TRƯỜNG ĐẠI HỌC AN GIANG
KHOA CÔNG NGHỆ THÔNG TIN

THỰC TẬP CUỐI KHÓA NGÀNH CÔNG NGHỆ THÔNG TIN

XÂY DỰNG HỆ THỐNG TRUNG TÂM HÓA ĐƠN ĐIỆN TỬ (EINVOICEHUB) VÀ PHÁT TRIỂN NỀN TẢNG TRIỂN KHAI LIÊN TỤC (CI/CD) TỰ ĐỘNG HÓA SỬ DỤNG GITOPS, KUBERNETES VÀ TERRAFORM

SINH VIÊN

DTH225650 – Huỳnh Quốc Huy

CÁN BỘ HƯỚNG DẪN

ThS. Nguyễn Thái Dư

AN GIANG, 01, 2026

TRƯỜNG ĐẠI HỌC AN GIANG
KHOA CÔNG NGHỆ THÔNG TIN

THỰC TẬP CUỐI KHÓA NGÀNH CÔNG NGHỆ THÔNG TIN

XÂY DỰNG HỆ THỐNG TRUNG TÂM HÓA ĐƠN ĐIỆN TỬ (EINVOICEHUB) VÀ PHÁT TRIỂN NỀN TẢNG TRIỂN KHAI LIÊN TỤC (CI/CD) TỰ ĐỘNG HÓA SỬ DỤNG GITOPS, KUBERNETES VÀ TERRAFORM

CÔNG TY CỔ PHẦN SOFTZ

DTH225650 - HUỲNH QUỐC HUY

AN GIANG, 01, 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)

Nội dung nhận xét:

- **Đồng ý** hay **Không đồng ý** cho sinh viên báo cáo TTCK; Nếu không đồng ý cần ghi rõ lý do.
- Kết quả được so với yêu cầu;
- Ý kiến khác (nếu có)

MỤC LỤC

I. GIỚI THIỆU ĐỀ TÀI 6
1.1. LÝ DO CHỌN ĐỀ TÀI 6
1.1.1. Đặt vấn đề và tính cấp thiết của đề tài 6
1.2. MỤC TIÊU CỦA ĐỀ TÀI 6
1.2.1. Giải quyết các vấn đề cấp thiết 6
1.2.2. Hiệu quả mang lại 8
1.3. PHẠM VI VÀ ĐỐI TƯỢNG NGHIÊN CỨU 8
1.3.1. Phạm vi các Module chức năng thực hiện và giới hạn các chức năng không thực hiện 8
1.3.2. Đối tượng nghiên cứu 10
II. TỔNG QUAN HỆ THỐNG 11
2.1. MÔ TẢ CHUNG HỆ THỐNG 11
2.1.1. Khái niệm trung tâm hoá đơn điện tử EInvoiceHub 11
2.1.2. Nghiệp vụ hóa đơn 13
2.1.3. Bối cảnh cảnh dự án và thực trạng chuyển đổi số tại cơ quan thực tập 14
2.2. ĐỐI TƯỢNG SỬ DỤNG 15
2.2.1. Nhóm Quản trị doanh nghiệp - Merchant Admin 15
2.2.2. Nhóm nhân viên vận hành và kế toán - Staff 15
2.2.3. Nhóm hệ thống tích hợp - API Client 16
2.2.4. Quản trị viên hệ thống - System Admin 16
2.3. PHẠM VI ỨNG DỤNG 16
2.3.1. Khả năng triển khai đa quy mô và đa doanh nghiệp 16
2.3.2. Ứng dụng đa lĩnh vực và ngành nghề kinh doanh 17
2.3.3. Đáp ứng tính linh hoạt về hạ tầng và pháp lý 17
III. PHÂN TÍCH YÊU CẦU HỆ THỐNG 17
3.1. YÊU CẦU CHỨC NĂNG 17
3.1.1. Các chức năng chính của hệ thống 17
3.1.2. Phân quyền người dùng 18
3.2. YÊU CẦU PHI CHỨC NĂNG 18
3.2.1. Giao diện thân thiện và trải nghiệm người dùng (UI/UX) 18
3.2.2. Hiệu năng và độ ổn định 19

3.2.3. Yêu cầu bảo mật và an toàn dữ liệu 19
IV. THIẾT KẾ HỆ THỐNG 19
4.1. Thiết kể tổng thể hệ thống 19
4.1.1. Kiến trúc phân tầng Layered Architecture 20
4.1.2. Mô hình Middleware Hub và Adapter Pattern 20
4.2. Thiết kế cơ sở dữ liệu 21
4.2.1. Chiến lược lưu trữ MariaDB tập trung 21
4.2.2. Sơ đồ thực thể quan hệ (ERD) 22
4.3. Thiết kế giao diện 22
4.4. Thiết kế hệ thống UML 22
4.4.1. Sơ đồ Trường hợp sử dụng - Use Case Diagram 22
4.4.2. Sơ đồ Lớp – Class Diagram 29
4.4.3. Sơ đồ Tuần tự - Sequences Diagram 33
4.4.4. Sơ đồ trạng thái ..... Error! Bookmark not defined.
4.4.5. Sơ đồ hoạt động 42
V. CÔNG NGHỆ VÀ MÔI TRƯỜNG PHÁT TRIỂN 42
5.1. Ngôn ngữ lập trình và Framwork 42
5.1.1. Java 21 – SpringBoot Framework 43
5.1.2. Type Script – Angolar Framework 44
5.2. Công cụ và môi trường phát triển 46
5.2.1. IntelliJ Idea 46
5.2.2. Visual Studio Code 47
5.2.3. Git và GitHub 47
5.2.4. Ubuntu 48
5.2.5. Docker 49
5.3. HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU 50
5.3.1. MariaDB 50
5.4. NỀN TẢNG TRIỂN KHAI VÀ TỰ ĐỘNG HÓA 51
5.4.1. Công nghệ Containerization Docker 51
5.4.2. Điều phối hạ tầng Kubernetes và Terraform 52
5.4.3. Quy trình triển khai hiện đại CI/CD và Triết lý GitOps với ArgoCD 54
VI. TỔNG KẾT 56

6.1. KẾT LUẬN 56

6.1.1. Hoàn thành mục tiêu xây dựng giải pháp Middleware tập trung 56

6.1.2. Hiệu quả của việc ứng dụng công nghệ Cloud-native và Java 21 56

6.2. HẠN CHẾ 56

6.2.1. Giới hạn về số lượng tích hợp nhà cung cấp và môi trường thử nghiệm 57

6.2.2. Khó khăn trong việc tương tác với các thiết bị ký số vật lý 57

6.3. HƯỚNG PHÁT TRIỂN 57

6.3.1. Mở rộng hệ sinh thái tích hợp ứng dụng trí tuệ nhân tạo 57

6.3.2. Nâng cấp kiến trúc Microservices và bảo mật bằng Blockchain 57

VII. TÀI LIỆU THAM KHẢO 58

DANH SÁCH HÌNH

Hình 1: Trang tài liệu IBM định nghĩa về Middleware 7
Hình 2: Tài liệu MuleSoft giải thích về API 12
Hình 3: Sơ đồ Trường hợp sử dụng tổng quát 23
Hình 4: Sơ đồ trường hợp sử dụng chi tiết 23
Hình 5: Sơ đồ Trường hợp sử dụng phân rã Use Case quản lý tài khoản định danh và quản lý cấu hình hệ thống 25
Hình 6: Sơ đồ Trường hợp sử dụng phân rã Use Case phát hành hóa đơn .. 26
Hình 7: Sơ đồ Trường hợp sử dụng phân rã Use Case phát hành hóa đơn và tra cứu và khai thác hóa đơn 26
Hình 8: Sơ đồ Trường hợp sử dụng phân rã Use Case quản lý tài khoản định danh và thiết lập dải số và mẫu hóa đơn 27
Hình 9: Sơ đồ Trường hợp sử dụng phân rã Use Case phát hành hóa đơn .. 28
Hình 10: Sơ đồ Trường hợp sử dụng phân rã Use Case xử lý nghiệp vụ sau phát hành 28
Hình 11: Sơ đồ Trường hợp sử dụng phân rã Use Case quản lý cấu hình và hệ thống 29
Hình 12: Sơ đồ Lớp tổng quát 30
Hình 13: Sơ đồ Tuần tự quy trình xuất hóa đơn 34
Hình 14: Sơ đồ Tuần tự quy trình ký hóa đơn 37
Hình 15: Sơ đồ Tuần tự quy trình lấy thông tin hóa đơn 40
Hình 16: Tài liệu Oracle về Java 21 43
Hình 17: Trang tài liệu Spring Boot 44
Hình 18: Trang tài liệu về TypeScript 45
Hình 19: Trang tài liệu Angular 46
Hình 20: Tài liệu Git và GitHub 47
Hình 21: Trang chủ Ubuntu 48
Hình 22: Trang tài liệu Docker 49
Hình 23: Trang tài liệu MariaDB Server 50
Hình 24: Trang tài liệu Docker giải thích về Docker Container và Docker Container Image 52
Hình 25: Trang tài liệu Kubernetes 53
Hình 26: Trang tài liệu HashiCorp giải thích về Cơ sở hạ tầng dưới dạng mã 53
Hình 27: Minh họa quy trình CI/CD 54

Hình 28: Trang tài liệu GitLab 55
Hình 29: Trang tài liệu Argo 55

DANH SÁCH BẢNG

Bảng 1: Bảng các thành phần tham gia quy trình xuất hóa đơn..... 35
Bảng 2: Bảng các thành phần tham gia quy trình ký hóa đơn..... 38
Bảng 3: Bảng các thành phần tham gia quy trình lấy thông tin hóa đơn 40

I. GIỚI THIỆU ĐỀ TÀI

1.1. LÝ DO CHỌN ĐỀ TÀI

1.1.1. Đặt vấn đề và tính cấp thiết của đề tài

1.1.1.1. Thực trạng phân mảnh và rào cản tích hợp hệ thống hóa đơn điện tử

Trong bối cảnh thực hiện chuyển đổi số toàn diện theo Thông tư 78/2021/TT-BTC và Nghị định 123/2020/NĐ-CP, 100% doanh nghiệp Việt Nam đã chuyển sang sử dụng hóa đơn điện tử. Tuy nhiên, thực trạng hiện nay cho thấy một bài toán nan giải là sự phân mảnh từ các nhà cung cấp dịch vụ hóa đơn (TVAN) như MISA, BKAV, Viettel hay VPNT. Mỗi nhà cung cấp sở hữu một giao diện lập trình ứng dụng (API) riêng biệt, chuẩn dữ liệu và quy trình kết nối hoàn toàn khác nhau. Điều này gây ra khó khăn cực lớn cho doanh nghiệp sở hữu nhiều hệ thống ERP, POS hay CRM khi muốn tích hợp hóa đơn, dẫn đến chi phí phát triển tăng cao và khả năng quản lý tập trung bị hạn chế. Việc thiếu tính đồng nhất trong giao tiếp dữ liệu giữa các bên cung cấp dịch vụ trung gian đã vô tình tạo ra các rào cản kỹ thuật, khiến cho quy trình tự động hóa dòng dữ liệu tài chính của doanh nghiệp trở nên rời rạc và khó kiểm soát

1.1.1.2. Nhu cầu cấp thiết về một nền tảng Middleware hiện đại và tự động

Động lực để triển khai đề tài xuất phát từ nhu cầu sở hữu một hệ thống trung tâm có khả năng chuẩn hóa toàn bộ luồng nghiệp vụ hóa đơn. Doanh nghiệp không chỉ cần một phần mềm xuất hóa đơn thuần, mà cần một giải pháp có tính sẵn sàng cao, khả năng chịu tải lơn và quy trình vận hành tự động. Để giải quyết bài toán hiệu năng xử lý, công nghệ Virtual Threads trong Java 21 được xem là một bước ngoặt. Việc xây dựng EinvoiceHub tích hợp các công nghệ tiên tiến như Java Virtual Threads giúp hỗ trợ xử lý đồng thời cực lớn và mô hình GitOps trên nền tảng Kubernetes là lời giải cho bài toán hiện đại hóa hạ tầng tài chính, đảm bảo tính pháp lý liên tục và tối ưu hóa nguồn lực cho doanh nghiệp trong kỷ nguyên số. Bên cạnh đó, việc quản trị hạ tầng theo phương thức hiện đại Infrastructure as Code (IaC) cũng là yếu tố then chốt. Sự kết hợp này không chỉ đảm bảo tính pháp lý liên tục mà còn tối ưu hóa nguồn lực cho doanh nghiệp trong kỷ nguyên số, giúp hệ thống không chỉ ổn định về mặt nghiệp vụ mà còn mạnh mẽ về mặt hạ tầng vận hành.

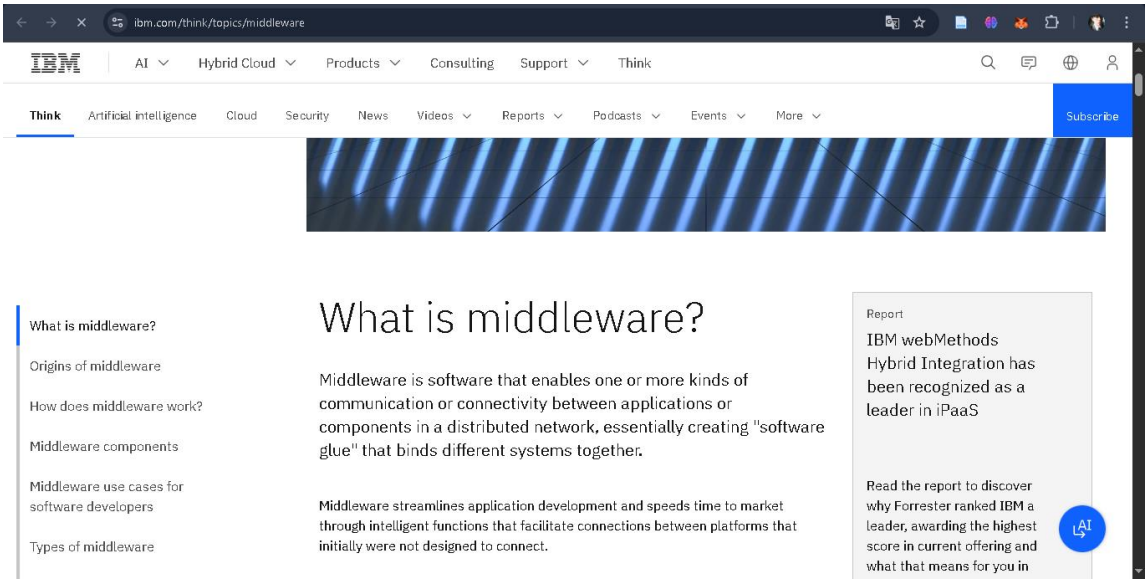
1.2. MỤC TIÊU CỦA ĐỀ TÀI

1.2.1. Giải quyết các vấn đề cấp thiết

1.2.1.1. Chuẩn hóa giao tiếp đa nền tảng và quản lý dữ liệu tập trung

Vấn đề cốt lõi mà dự án tập trung giải quyết là xây dựng một lớp trừu tượng hóa Abstraction Layer sử dụng Strategy và Adapter Pattern. Hệ thống sẽ đứng ra tiếp nhận mọi yêu cầu phát hành hóa đơn theo một định dạng chuẩn duy nhất, sau đó tự động điều phối và chuyển đổi dữ liệu phù hợp với từng nhà cung cấp (TVAN) cụ thể. Ngoài ra, dự án giải quyết bài toán quản lý tập trung chứng thư số (Certificate) , hạn mức hóa đơn (Quota) và lưu trữ Payload (XML/JSON) tại một cơ sở dữ liệu duy nhất MariaDB, giúp doanh nghiệp có cái nhìn tổng thể về tài chính mà không cần truy cập lẻ tẻ vào từng hệ thống của nhà cung cấp.

Giải pháp này đóng vai trò như một lớp trung gian kỹ thuật quan trọng. Theo định nghĩa từ IBM: Middleware là phần mềm cho phép một hoặc nhiều loại giao tiếp hoặc kết nối giữa các ứng dụng hoặc thành phần trong một mạng phân tán, về cơ bản tạo ra "chất kết dính phần mềm" liên kết các hệ thống khác nhau lại với nhau. Phần mềm trung gian giúp đơn giản hóa quá trình phát triển ứng dụng và rút ngắn thời gian đưa sản phẩm ra thị trường thông qua các chức năng thông minh, tạo điều kiện kết nối giữa các nền tảng ban đầu không được thiết kế để kết nối với nhau.[1]. Áp dụng mô hình này giúp EInvoiceHub trở thành hạt nhân điều phối, đảm bảo tính thống nhất về dữ liệu bất kể doanh nghiệp đang sử dụng dịch vụ của MISA, BKAV hay các nhà cung cấp khác.



Hình 1: Trang tài liệu IBM định nghĩa về Middleware

1.2.1.2. Tự động hóa vận hành và đảm bảo tính ổn định hạ tầng

Dự án tập trung xử lý vấn đề rủi ro trong triển khai phần mềm thủ công. Thông quan việc xây dựng luồng CI/CD tự động hóa hoàn toàn và ứng dụng triết lý GitOps (ArgoCD), mọi thay đổi về mã nguồn hay cấu hình hạ tầng đều được kiểm soát phiên bản chặt chẽ trên Git. Hệ thống giải quyết bài toán tự phục hồi và co giãn linh hoạt trên cụm Kubernetes đảm bảo dịch vụ hóa đơn không bị gián đoạn ngay cả khi có sự cố hạ tầng hoặc lưu lượng giao dịch tăng đột biến vào các thời điểm quyết toán thuế.

1.2.2. Hiệu quả mang lại

1.2.2.1. Tối ưu hóa chi phí và nâng cao năng suất nghiệp vụ

Về mặt kinh tế, EinvoiceHub giúp giảm thiểu tới 70% thời gian và chi phí tích hợp cho doanh nghiệp khi muốn thay đổi hoặc sử dụng đồng thời nhiều nhà cung cấp hóa đơn. Việc sử dụng Java 21 Virtual Threads cho phép hệ thống xử lý hàng nghìn giao dịch ký số và truyền nhận API đồng thời với mức tiêu tốn tài nguyên phần cứng cực thấp, tối ưu hóa chi phí vận hành máy chủ. Đối với bộ phận kế toán, hệ thống mang lại sự chính xác tuyệt đối trong việc theo dõi trạng thái hóa đơn và mã cấp từ Cơ quan Thuế theo thời gian thực.

1.2.2.2. Chuyên nghiệp hóa hạ tầng công nghệ thông tin

Dự án mang lại hiệu quả lớn về công nghệ là chuyển đổi mô hình vận hành từ truyền thống sang Cloud-Native. Việc triển khai thành công mô hình GitOps và quản lý hạ tầng bằng mã nguồn Terraform giúp doanh nghiệp sở hữu một quy trình phát hành phần mềm tiêu chuẩn quốc tế. Hệ thống không chỉ đảm bảo tính bảo mật dữ liệu tài chính qua các lớp Ingress Controller và SSL/TLS, mà còn cung cấp khả năng quan sát toàn diện qua Prometheus và Grafana, giúp đội ngũ kỹ thuật chủ động kiểm soát hệ thống, mang lại sự tin tưởng tuyệt đối cho khách hàng và đối tác.

1.3. PHẠM VI VÀ ĐỐI TƯỢNG NGHIÊN CỨU

1.3.1. Phạm vi các Module chức năng thực hiện và giới hạn các chức năng không thực hiện

1.3.1.1. Module nghiệp vụ hóa đơn cốt lõi

Đây là module trọng tâm xử lý vòng đời của hóa đơn, trong đó bao gồm các chức năng khởi tạo và xuất hóa đơn, ký hóa đơn điện tử, truy xuất và lấy thông tin hóa đơn.Với chức năng khởi tạo và xuất hóa đơn hay Invoice Issuance có nhiệm vụ tiếp nhận dữ liệu hô từ các hệ thống ERP/POS, sau đó thực hiện kiểm tra hạn mức hay Quota, sau đó đối soát

dải số và khởi tạo hóa đơn dưới dạng bản nháp. Sử dụng cấu trúc dữ liệu chuẩn XML theo quy định của Tổng Cục Thuế.

Bên cạnh đó là chức năng ký hóa đơn điện tử hay Digital Signing bằng cách tích hợp quy trình ký sô sử dụng chuẩn XMLDSig. Hệ thống hỗ trợ cơ chế ký thông quan Cloud HSM dịch vụ cho phép ký lô lớn và tự động và USB Token hay ký thủ công qua đó đảm bảo tính toàn vẹn và giá trị pháp lý cho hóa đơn .Ngoài ra chức năng không kém quan trọng là truy xuất và lấy thông tin hóa đơn hay Invoice Retrieval từ việc xây dựng bộ lọc tìm kiếm thông minh theo mã định danh nội bộ hóa đơn hoặc mã cấp từ Cơ quan Thuế. Hệ thống sẽ hỗ trợ trịch xuất dữ liệu gốc XML và hiển thị bản thể hiện trực quan PDF thông qua việc kết nối API tới các nhà cung cấp.

1.3.1.2. Module quản trị và kết nối hạ tầng

Hệ thống cung cấp chức năng quản trị Merchant và cấu hình Adapter bằng việc xây dựng hệ thống quản lý đa doanh nghiệp, cho phép thiết lập linh hoạt các Adapter kết nối tới các nhà cung cấp khác như MISA, BKAV, Viettel… thông qua Strategy Pattern. Kế đến là khả năng tự động hóa triển khai (CI/CD vaf GitOps) từ thiết lập Pipeline tự động hóa từ khâu kiểm thử, đóng gói Docker Image đến khâu triển khai đồng bộ lên cụm Kubernetes thông quan ArgoCD. Sử dụng Terraform để quản lý hạ tầng dưới dạng mã (IaC).

1.3.1.3. Giới hạn về pháp lý và tổ chức cung cấp dịch vụ

Hệ thống không đóng vai trò TVAN mà hoạt động như một lớp trung gian. Đề tài không thực hiện việc trực tiếp truyền nhận dữ liệu với cổng của Tổng cục Thuế mà thông qua API của các tổ chức cung cấp dịch vụ hóa đơn (TVAN) đã được cấp phép.Ngoài ra hệ thống cũng không cấp phát Chứng thư số (CA) đề tài chỉ thực hiện tích hợp và sử dụng các chứng thư số có sẵn, không xây dựng hạ tầng cấp phát hay xác thực chứng thư số công cộng.

1.3.1.4. Giới hạn về mặt kỹ thuật và quy mô triển khai

Số lượng Adapter kết nối trong khuôn khổ thực tập, hệ thống sẽ ưu tiên hoàn thiện Adapter kết nối cho 2 – 3 nhà cung cấp phổ biến nhất như MISA BKAV,… để chứng minh khả năng hoạt động của Strategy Pattern thay vì tích hợp toàn bộ tất cả các nhà cung cấp trên thị trường. Thêm vào đó các giao dịch ký số và cấp mã hóa đơn được thực hiện trên môi trường Demo thử nghiệm của các nhà cung cấp để đảm bảo tính an toàn

dữ liệu và chi phí triển khai. Tính thực tiễn của việc tích hợp này được khẳng định qua khả năng hỗ trợ đa dạng các loại nghiệp vụ phát sinh theo quy định mới nhất.

1.3.2. Đối tượng nghiên cứu

1.3.2.1. Kiến trúc hệ thống trung gian Middleware và thiết kế mẫu Design Patterns

Đề tài nghiên cứu cách xây dựng một kiến trúc Middleware, đây là một lớp Hub trung gian có khả năng trừu tượng hóa sự phức tạp các nhà cung cấp hóa đơn khác nhau. Đồng thời đối tượng của đề tài hướng đến còn là việc áp dụng các mẫu thiết kế dựa trên mô hình Strategy và Adapter Pattern để đóng gói logic riêng biệt của từng Provider như MISA, BKAV, Viettel…Điều này cho phép hệ thống mở rộng kết nối với các nhà cung cấp mới mà không cần can thiệp vào mã nguồn lõi của nghiệp vụ phát hành hóa đơn.

1.3.2.2. Chiến lược lưu trữ dữ liệu nhất quán và hiệu năng

Nghiên cứu chiến lược lưu trữ tập trung trên cơ sở dữ liệu quan hệ MariaDB . Thay vì sử dụng NoSQL thì đề tài tập trung tối ưu hóa việc quản lý đồng thời hai loại dữ liệu nghiệp vụ và kỹ thuật. Dữ liệu nghiệp vụ hóa đơn bao gồm các thông tin định danh trạng thái và tài chính cần tính nhất quán trong hệ quản trị cơ sở dữ liệu (ACID) tuyệt đối. Dữ liệu kỹ thuật lưu trữ các tệp XML và JSON lớn dưới dạng LONGTEXT, nghiên cứu kỹ thuật Indexing trên các trường định danh mã tra cứu và Partitioning theo thời gian để đảm bảo tốc độ truy vấn khi dữ liệu đạt quy mô hàng triệu bản ghi.

Bên cạnh đó đề tài cũng nghiên cứu ứng dụng của luồng ảo Virtual Threads trong Java 21 để giải quyết bài toán bài toán nghẽn luông I/O. Đối tượng nghiên cứu là khả năng xử lý hàng nghìn kết nối đồng thời tới API của các nhà cung cấp hóa đơn mà vẫn giảm mức tiêu thụ tài nguyên RAM và CPU.

1.3.2.3. Giải pháp bảo mật và tuân thủ pháp lý

Nghiên cứu hướng đến cấu trúc XML theo quy định Tổng cục thuế và cách thức nhúng chữ ký số đảm bảo tính toàn vẹn và không thể chối bỏ. Bên cạnh đó là việc tích hợp quy trình ký số tự động qua Cloud HSM cho doanh nghiệp xuất hóa đơn theo lô lớn và ký thủ công qua USB Token Driver.

1.3.2.4. Mô hình vận hành GitOps và hạ tầng

Đối tượng của đề tài còn là việc định nghĩa tài nguyên hệ thống Mạng, cụm Kubernetes, Database hoàn toàn bằng mã nguồn, đảm bảo tính tái sử dụng và khả năng triển khai nhanh thông quan quản lý hạ tầng bằng Terraform.

Với mô hình GitOps kết hợp ArgoCD trên Kubernete, nghiên cứu hướng đến quy trình đồng bộ hóa trạng thái liên tục mà trọng tâm là việc lấy Git Repository là nguồn duy nhất và nghiên cứu cách ArgoCD tự động phát hiện và cập nhật các thay đổi cấu hình từ Git lên Cụm Kubernetes, đảm bảo môi trường Production luôn đồng nhất và có khả năng tự phục hồi.

II. TỔNG QUAN HỆ THỐNG

2.1. MÔ TẢ CHUNG HỆ THỐNG

2.1.1. Khái niệm trung tâm hoá đơn điện tử EInvoiceHub

2.1.1.1. Khái niệm về Middleware và vai trò kết nối Trung tâm.

EinvoiceHub là một hệ thống trung gian Middleware đóng và trò là hạt nhân trong hệ sinh thái tài chính số của doanh nghiệp. Thay vì các ứng dụng đầu cuối như ERP (Enterprise Resource Planning), CRM hay POS (Point of Sale) phải thực hiện kết nối riêng lẻ, phức tạp với từng nhà cung cấp dịch vụ hóa đơn, EinvoiceHub cung cấp một cổng giao tiếp tập trung. Hệ thống chịu trách nhiệm chuẩn hóa dữ liệu, điều phối luồng nghiệp vụ và đảm bảo tính đồng bộ giữa các hệ thống quản trị nội bộ với cơ sở dữ liệu của nhà cung cấp và cơ quan thuế.

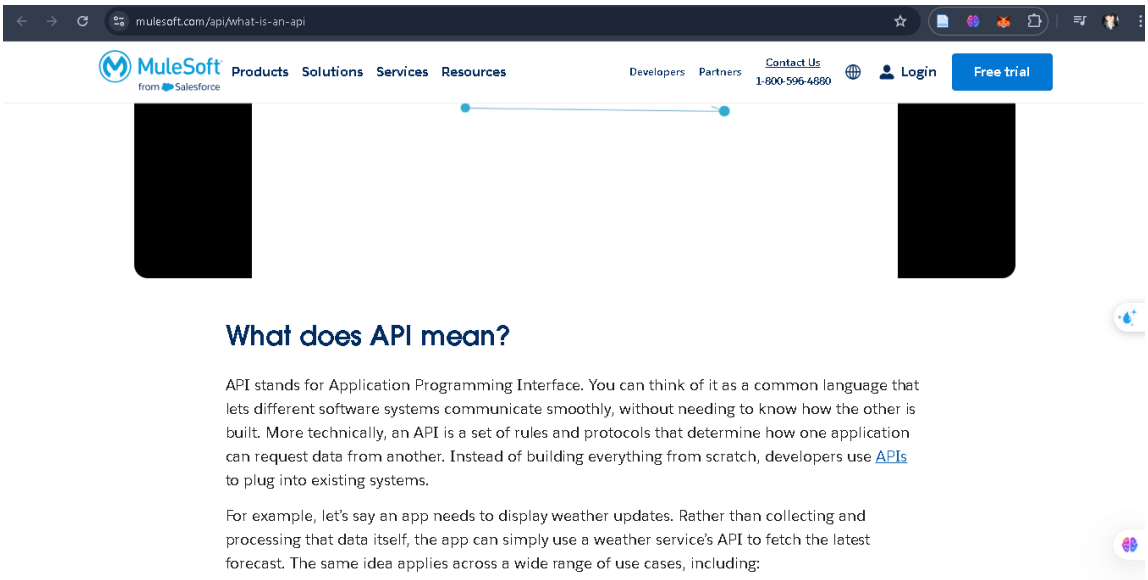
Về mặt kỹ thuật, vai trò cốt lõi của một hệ thống trung gian đã được tổ chức Red Hat định nghĩa rõ nét nhằm khẳng định tính tất yếu của nó trong kiến trúc phần mềm: Middleware là một lớp phần mềm kết nối hệ điều hành với các ứng dụng, dữ liệu và người dùng. Nó cung cấp các dịch vụ và khả năng chung, chẳng hạn như đăng nhập một lần (SSO) hoặc quản lý giao diện lập trình ứng dụng (API) . Các nhà phát triển có thể dựa vào middleware để cung cấp sự tích hợp nhất quán và đơn giản giữa các thành phần ứng dụng. Điều này giúp các nhà phát triển tập trung vào việc xây dựng các tính năng cốt lõi của ứng dụng, thay vì dành thời gian kết nối các tính năng đó với các điểm cuối và môi trường khác nhau, bao gồm cả các hệ thống cũ.[2]

2.1.1.2. Tính chuẩn hóa và khả năng mở rộng

Một trong những khái niệm cốt lõi của EInvoiceHub là đồng nhất trong đa dạng. Hệ thống được thiết kế dựa trên các kiến trúc phần mềm tiên tiến, cho phép trừu tượng hóa logic của nhiều nhà cung cấp hóa đơn khác nhau (như MISA, BKAV, Viettel, VNPT) thành một bộ giao diện

hay API duy nhất. Điều này giúp doanh nghiệp linh hoạt trong việc thay đổi hoặc sử dụng đồng thời nhiều nhà cung cấp mà không cần thay đổi cấu trúc của các hệ thống, đồng thời tối ưu hóa việc quản lý chứng thư số và dải số hóa đơn tại một điểm duy nhất

Cơ sở của tính chuẩn hóa này bắt nguồn từ việc tận dụng sức mạnh của giao diện lập trình ứng dụng hiện đại. Theo MuleSoft (thuộc Salesforce): API là viết tắt của Application Programming Interface (Giao diện Lập trình Ứng dụng). Bạn có thể coi nó như một ngôn ngữ chung cho phép các hệ thống phần mềm khác nhau giao tiếp trơn tru mà không cần biết hệ thống kia được xây dựng như thế nào. Về mặt kỹ thuật hơn, API là một tập hợp các quy tắc và giao thức xác định cách một ứng dụng có thể yêu cầu dữ liệu từ một ứng dụng khác. Thay vì xây dựng mọi thứ từ đầu, các nhà phát triển sử dụng API để tích hợp vào các hệ thống hiện có.[3]. Nhờ áp dụng mô hình này, EInvoiceHub giúp xóa bỏ sự khác biệt về giao thức kỹ thuật của từng nhà cung cấp, tạo ra một môi trường tích hợp đồng nhất cho doanh nghiệp.



Hình 2: Tài liệu MuleSoft giải thích về API

2.1.1.3. Nền tảng Cloud-Native và tự động hóa vận hành

Vượt xa một ứng dụng nghiệp vụ truyền thống, EinvoiceHub tại diện cho xu hướng phần mềm hiện địa thông qua đặc tính của Cloud-Native. Hệ thống được xây dựng để vận hành trên nền tảng Container qua sử dụng Docker và điều phối bởi Kubernetes. Việc ứng dụng triết lý của GitOpts và cơ chế hạ tầng Infrastructure as Code (IaC) giúp cho EinvoiceHub có khả năng tự phục hồi sau sự có và mở rộng linh hoạt theo tải trọng và triển khai hoàn toàn tự động. Đây không chỉ là một trung

tâm dữ liệu hóa đơn mà còn là một quy trình vận hành công nghệ tiêu chuẩn, đảm bảo tính sẵn sàng cao nhất cho các giao dịch tài chính trọng yếu.

Đặc tính này hoàn toàn tương thích với định nghĩa về công nghệ hiện đại của tổ chức Cloud Native Computing Foundation (CNCF): Các công nghệ điện toán đám mây gốc cho phép các tổ chức xây dựng và vận hành các ứng dụng có khả năng mở rộng trong các môi trường hiện đại, năng động như đám mây công cộng, riêng tư và lai. Container, service mesh, microservices, cơ sở hạ tầng bất biến và API khai báo là những ví dụ điển hình cho cách tiếp cận này. [4]. Việc tuân thủ các nguyên tắc này cho phép EInvoiceHub đạt được sự bền bỉ và tốc độ triển khai vượt trội so với các hệ thống quản trị hóa đơn truyền thống.

2.1.2. Nghiệp vụ hóa đơn

2.1.2.1. Định nghĩa hóa đơn giá trị gia tăng

Hoá đơn giá trị gia tăng được định nghĩa tại khoản 1 Điều 8 Nghị định 123/2020/NĐ-CP có thể hiểu là hóa đơn dành cho các tổ chức khai thuế giá trị gia tăng theo phương pháp khấu trừ sử dụng cho các hoạt động:

Bán hàng hóa, cung cấp dịch vụ trong nội địa;

Hoạt động vận tải quốc tế;

Xuất vào khu phi thuế quan và các trường hợp được coi như xuất khẩu;

Xuất khẩu hàng hóa, cung cấp dịch vụ ra nước ngoài. [5]

2.1.2.2. Định nghĩa hóa đơn điện tử

Hóa đơn điện tử là hóa đơn có mã hoặc không có mã của cơ quan thuế được thể hiện ở dạng dữ liệu điện tử do tổ chức, cá nhân bán hàng hóa, cung cấp dịch vụ lập bằng phương tiện điện tử để ghi nhận thông tin bán hàng hóa, cung cấp dịch vụ theo quy định của pháp luật về kế toán, pháp luật về thuế, bao gồm cả trường hợp hóa đơn được khởi tạo từ máy tính tiền có kết nối chuyển dữ liệu điện tử với cơ quan thuế, trong đó:

a) Hóa đơn điện tử có mã của cơ quan thuế là hóa đơn điện tử được cơ quan thuế cấp mã trước khi tổ chức, cá nhân bán hàng hóa, cung cấp dịch vụ gửi cho người mua
b) Mã của cơ quan thuế trên hóa đơn điện tử bao gồm số giao dịch là một dãy số duy nhất do hệ thống của cơ quan thuế tạo ra và một chuỗi ký tự được cơ quan thuế mã hóa dựa trên thông tin của người bán lập trên hóa đơn.

c) Hóa đơn điện tử không có mã của cơ quan thuế là hóa đơn điện tử do tổ chức bán hàng hóa, cung cấp dịch vụ gửi cho người mua không có mã của cơ quan thuế.[5]

2.1.2.3. Định nghĩa hóa đơn từ máy tính tiền

Theo khoản 2, Điều 1, Nghị định số 70/2025/NĐ-CP được sửa đổi, bổ sung Điểm c, Khoản 2, Điều 3, Nghị định số 123/2020/NĐ-CP định nghĩa:

"c) Hóa đơn điện tử khởi tạo từ máy tính tiền có kết nối chuyển dữ liệu điện tử với cơ quan thuế (sau đây gọi là hóa đơn điện tử khởi tạo từ máy tính tiền) là hóa đơn có mã của cơ quan thuế hoặc dữ liệu điện tử để người mua có thể truy xuất, kê khai thông tin hóa đơn điện tử khởi tạo từ máy tính tiền do tổ chức, cá nhân bán hàng hóa, cung cấp dịch vụ lập từ hệ thống tính tiền, dữ liệu được chuyển đến cơ quan thuế theo định dạng được quy định tại Điều 12 Nghị định này.

d) Máy tính tiền là hệ thống tính tiền bao gồm một thiết bị điện tử đồng bộ hoặc một hệ thống gồm nhiều thiết bị điện tử được kết hợp với nhau bằng giải pháp công nghệ thông tin có chức năng chung như: tính tiền, lưu trữ các giao dịch bán hàng, số liệu bán hàng." [6]

2.1.3. Bối cảnh cảnh dự án và thực trạng chuyển đổi số tại cơ quan thực tập

2.1.3.1. Tác động của pháp lý và nhu cầu số hóa toàn diện

Dự án triển khai trong bối cảnh Chính phủ Việt nam đẩy mạnh lộ trình chuyển đổi số quốc gia, đặc biệt là việc áp dụng bắt buộc hóa đơn điện tử theo Thông tư 78/2021/TT-BTC và Nghị định 123/2020/NĐ-CP. Tại cơ quan thực tập, nhu cầu về một hệ thống quản trị minh bạch, chính xác và có khả năng tích hợp cao là một nhu cầu cấp thiết. Thực trạng tại đơn vị cho thấy việc xử lý hóa đơn thủ cộng và tích hợp rời rạc đang gây ra những bất lợi lớn về tốc độ xử lý, chi phí vận hành và rủi ro sai sót trong báo cáo thuế định kỳ.

2.1.3.2. Sự chuyển dịch từ Hệ thống đơn lẻ sang mô hình Hub tập trung

Trước khi triển khai dự án, thực trạng công nghệ tại đơn vị đang đối mặt với bài toán phân mảnh dữ diệu khi phải quản trị nhiều tài khoản từ các nhà cung cấp khác nhau một cách thủ công.Việc thiếu một công cụ kiểm soát hạn mức (Quota) và theo dõi trạng thái hóa đơn tập trung đã thúc đẩy ý tưởng xây dựng EinvoiceHub. Dự án triển khai không chỉ giải quyết bài toán nghiệp vụ hiện hành mà còn được xem là bước đi chiến lược nhằm hiện đại hóa hạ tầng công nghệ thông tin của đơn vị, áp dụng những công nghệ hàng đầu như Java Virtual Threads và GitOps.

2.2. ĐỐI TƯỢNG SỬ DỤNG

Hệ thống EinvoiceHub được thiết kế phân quyền chặt chẽ nhằm đảm bảo tính bảo mật và chuyên môn hóa trong quy trình quản lý hóa đơn. Người dùng được chia làm bốn nhóm chính với các vai trò và phạm vi tác động khác nhau.

2.2.1. Nhóm Quản trị doanh nghiệp - Merchant Admin

Đây là nhóm người dùng có quyền cao nhất trong phạm vi một doanh nghiệp sử dụng dịch vụ trên Hub. Họ chịu trách nhiệm thiết lập nền tảng để hệ thống có thể vận hành.

Nhóm này có quyền quản lý người dùng nội bộ, từ thực hiện thêm mới, chỉnh sửa, phân quyền hoặc vô hiệu hóa tài khoản nhân viên Staff trong doanh nghiệp.

Nhóm quản trị doanh nghiệp có thể cấu hình kết nối nhà cung cấp bằng các chức năng như thiết lập thông tin tài khoản API, định danh kết nối với các TVAN như MISA, BKAV, Viettel… cho doanh nghiệp mình. Họ còn quản lý chứng thư số qua khai báo thông tin Serial, thời hạn của chữ ký số (HSM hoặc USB Token) dùng để ký hóa đơn.

Ngoài ra nhóm này có thể thiết lập dải số và mẫu hóa đơn thông qua cấu hình mẫu số, ký hiệu và đăng ký dải số hóa đơn theo thông báo phát hành đã được Cơ quan Thuế chấp nhận.

Chức năng cuối cùng của nhóm quản trị doanh nghiệp là họ có thể giám sát hạn mức từ việc theo dõi số lượng hóa đơn đã sử dụng và hạn mức còn lại để đảm bảo hoạt động kinh doanh không bị gián đoạn.

2.2.2. Nhóm nhân viên vận hành và kế toán - Staff

Đây là nhóm trực tiếp xử lý các nghiệp vụ háo đơn hàng ngày, tương tác với giao diện Web để thực hiện các thao tác thủ công.

Họ có quyền lập và phát hành hóa đơn qua việc khởi tạo dữ liệu hóa đơn từ bản nháp, kiểm tra thông tin và thực hiện lệnh ký số để phát hành.

Nhóm nhân viên còn có thể xử lý các nghiệp vụ sau phát hành từ việc thực hiện các quy trình điều chỉnh thông tin, thay thế hóa đơn sai sót hoặc hủy bỏ hóa đơn theo quy định của Thông tư 78/2021/TT-BTC.

Một chức năng mà nhóm này có thể sử dụng đó là khai thác và tra cứu. Cụ thể nhân viên có thể tìm kiếm hóa đơn theo nhiều tiêu chí như mã tra cứu, mã cơ quan thuế, khách hàng hoặc tải về bản gốc XML hoặc bản thể hiện PDF để gửi cho khách hàng.

Ngoài ra nhân viên có thể quản lý cáu hình danh mục hàng hóa dịch vụ và khách hàng thường xuyên để tối ưu hóa tốc độ lập hóa đơn.

2.2.3. Nhóm hệ thống tích hợp - API Client

Đây là tác nhân hệ thống đại diện cho các phần mềm ERP, POS, CRM của doanh nghiệp kết nối với Hub thông qua giao thức REST API

Nhóm này giúp tự động hóa phát hành hóa đơn bằng cách đẩy dữ liệu bán hàng từ hệ thống gốc sang EinvoiceHub để thực hiện ký số và cấp mã CQT mà không cần con người can thiệp.

Đồng bộ trạng thái thời gian thực từ việc nhận các tín hiệu phản hồi từ Hub về trạng thái hóa đơn Thành công hay Thất bại để cập nhật ngược lại vào hệ thống kế toán nội bộ

Hệ thống tích hợp có thể truy xuất dữ liệu động nhờ gọi API để lấy thông tin hóa đơn, tếp dữ liệu XML phục vụ mục đích lưu trữ hoặc xử lý kế toán tự động.

2.2.4. Quản trị viên hệ thống - System Admin

Đây là nhóm kỹ sư DevOps chịu trách nhiệm về sự vận hành của toàn bộ nền tảng Hub, quản lý hạ tầng và quy trình triển khai phần mềm.

Quản trị hệ thống có thể quản lý nhóm quản trị doanh nghiệp - Merchant, từ khởi tạo tài khoản cho các doanh nghiệp mới tham gia hệ thống , thiết lập gói dịch vụ và cấp hạn mức hóa đơn tổng.

Quyền tiếp theo không kém quan trọng là giám sát hạ tầng Kubernetes nhờ theo dõi trạng thái hoạt động của các Pods, Services và Ingress trong cụm K8s để đảm bảo tính sẵn sàng cao.

Thêm vào đó quản trị viên hệ thống có quyền điều phối luồng GitOps từ quản lý cấu hình hạ tầng qua Terraform và giám sát việc đồng bộ hóa mã nguồn thông quan ArgoCD, đảm bảo mọi thay đổi trên Production đều khớp với cấu hình trong Git.

Chức năng đáng nói đến của quản trị viên hệ thống là quản trị bảo mật và Audit log bằng giám sát nhật ký hệ thống , quản lý các Secrets như API Key, Database Password và đảm bảo an toàn luồng dữ liệu truyền nhận giữ các bên.

2.3. PHẠM VI ỨNG DỤNG

2.3.1. Khả năng triển khai đa quy mô và đa doanh nghiệp

Hệ thống không chỉ phục vụ cho một doanh nghiệp đơn lẻ mà được xây dựng như một nền tảng hỗ trợ đa người dùng. Với doanh nghiệp vừa và nhỏ hệ thống cung cấp giao diện Web trực quan để kế toán thực hiện các thao tác lập và ký hóa đơn thủ công một cách nhanh chóng mà không cần đầu tư hạ tầng phức tạp. Với tập đoàn và Tổng công ty EinvoiceHub đáp ứng nhu cầu quản trị tập trung hóa đơn của nhiều đơn vị thành viên,

chi nhánh trên một hệ thống duy nhất, giúp đồng nhất dữ liệu tài chính toàn tập đoàn.

2.3.2. Ứng dụng đa lĩnh vực và ngành nghề kinh doanh

Nhờ tuân thủ nghiêm ngặt các chuẩn dữ liệu XML của Tổng cục Thuế và hỗ trợ đa dạng loại hóa đơn, hệ thống có thể áp dụng cho nhiều lĩnh vực khác nhau.

Cụ thể với lĩnh vực bán lẻ và F&B hệ thống tích hợp trực tiếp với máy tính tiền để xuất hóa đơn ngay khi kết thúc giao dịch cho hóa đơn khởi tạo từ máy tính tiền.

Bên cạnh đó lĩnh vực dịch vụ và thương mại cũng được hỗ trợ xử lý các loại hóa đơn giá trị gia tăng (VAT), hóa đơn bán hàng cho các hoạt động cung cấp dịch vụ nội địa và xuất khẩu.

Ngoài ra trong lĩnh vực thương mại điện tử hệ thống giúp tự động hóa hoàn toàn quy trình xuất hóa đơn cho hàng ngàn đơn hàng mỗi ngày thông qua kết nối API, giảm thiểu tối đa sai sót do con người.

2.3.3. Đáp ứng tính linh hoạt về hạ tầng và pháp lý

Hệ thống đáp ứng được tính linh hoạt hạ tầng nhờ công nghệ Cloud-Native (Docker/Kubernetes), hệ thống có thể triển khai trên cả hạ tầng điện toán đám mây (Public Cloud) hoặc hạ tầng riêng của doanh nghiệp (On-premise), tùy thuộc vào yêu cầu bảo mật dữ liệu.

Bên cạnh đó hệ thống còn tuân thủ pháp lý liên tục vì được thiết kế để dễ dàng cập nhật khi có sự thay đổi về chính sách thuế hoặc định dạng dữ liệu từ cơ quan Nhà nước mà không làm gián đoạn luồng vận hành của doanh nghiệp.

III. PHÂN TÍCH YÊU CẦU HỆ THỐNG

3.1. YÊU CẦU CHỨC NĂNG

3.1.1. Các chức năng chính của hệ thống

Hệ thống tập trung vào việc quản lý toàn bộ vòng đời của hóa đơn điện tử và tối ưu hóa kết nối đa nhà cung cấp.

Tính năng cốt lõi chính là quy trình phát hành hóa đơn. Trong đó hệ thống sẽ tiếp nhận dữ liệu hóa đơn từ các nguồn ERP/POS qua API, thực hiện kiểm tra tính hợp lệ của dữ liệu và đối soát hạn mức Quota trước khi khởi tạo bản nháp. Sau đó là tích hợp quy trình ký điện tử chuẩn XMLDSig. Hệ thống tự động điều phối lệnh ký tới Cloud HSM cho các yêu cầu số lượng lớn hoặc hỗ trợ ký qua USB Token cho các thao tác thủ công. Cuối cùng hệ thống sẽ tự động đóng gói dữ liệu theo chuẩn XML

của Tổng cục Thuế và gửi tới các tổ chức TVAN. Hệ thống theo dõi trạng thái truyền nhận và cập nhật mã cơ quan thuế ngay khi có kết quả.

Bên cạnh phát hành hóa đơn, hệ thống cần thiết kế để quản lý cấu hình và Adapter. Trong đó cho phép quản trị viên cấu hình thông số kết nối (URL, API Key, Secret,…) cho từng nhà cung cấp dịch vụ (MISA, BKAV...). Có thể áp dụng Strategy Pattern để linh hoạt chuyển đổi hoặc sử dụng đồng thời nhiều Provider trên cùng một hệ thống Hub. Hệ thống còn hỗ trợ khai báo và giám sát thời hạn của các chứng thư số, đảm bảo quy trình ký số không bị gián đoạn.

Hơn nữa hệ thống cần lưu trữ dữ liệu một cách có chiến lược. Mà MariaDB là database được sử dụng. Việc lưu trữ này cần đảm bảo tính nhất quán (ACID) cho dữ liệu nghiệp vụ và lưu trữ nội dung thô của hóa đơn dưới dạng XML/JSON phục vụ mục đích tra cứu đối soát và kết xuất báo cáo thuế định kỳ.

3.1.2. Phân quyền người dùng

Hệ thống áp dụng mô hình kiểm soát truy cập dựa trên vai trò Role-Based Access Control – RBAC để đảm bảo tính an toàn và minh bạch.

Merchant Admin hay Quản trị doanh nghiệp có toàn quyền trong phạm vi doanh nghiệp mình quản lý. Thực hiện các thao tác cấu hình hệ thống, quản lý người dùng nội bộ, thiết lập dải số hóa đơn và giám sát hạn mức sử dụng.

Staff là Nhân viên vận hành hoặc Kế toán có quyền thực hiện các nghiệp vụ trực tiếp trên giao diện Web như lập hóa đơn, ký số thủ công, xử lý các nghiệp vụ sai sót (hủy/điều chỉnh) và tra cứu, tải tệp PDF/XML hóa đơn.

API Client hay Hệ thống tích hợp đây chính là tác nhân hệ thống thực hiện đẩy dữ liệu tự động từ ERP/POS sang Hub thông qua API Key. Nhóm này chỉ có quyền thực hiện các lệnh liên quan đến phát hành và truy vấn trạng thái hóa đơn tự động.

System Admin đây là tác nhân có vai trò quản trị hệ thống Hub, họ chịu trách nhiệm vận hành hạ tầng Kubernetes, quản lý danh sách các Merchant, giám sát luồng GitOps và bảo trì các kết nối API mức hệ thống.

3.2. YÊU CẦU PHI CHỨC NĂNG

3.2.1. Giao diện thân thiện và trải nghiệm người dùng (UI/UX)

Giao diện cần đảm bảo tính trực quan qua thiết kế Dashboard trên Angular phải hiển thị rõ ràng các chỉ số quan trọng như số lượng hóa đơn đã xuất, hạn mức còn lại và trạng thái lỗi từ cơ quan Thuế.

Kế đến giao diện cần đảm bảo tính linh hoạt (Responsive Design) có thể hiển thị tối ưu trên đa dạng thiết bị Desktop, Tablet… giúp nhân viên kế toán có thể kiểm tra trạng thái hóa đơn mọi lúc, mọi nơi.

Một yêu cầu quan trọng chính là tốc độ phản hồi qua xử lý các thao tác trên giao diện người dùng phải được xử lý mượt mà, thời gian phản hồi cho các truy vấn danh sách hóa đơn không quá 2 giây.

3.2.2. Hiệu năng và độ ổn định

Hệ thống phải đảm bảo tốc độ xử lý đồng thời cao. Đây là yêu cầu quan trọng nhất. Hệ thống ứng dụng Java 21 Virtual Threads để xử lý hàng ngàn kết nối I/O cùng lúc khi gọi API đến các TVAN mà không gây nghẽn luồng hệ thống, đảm bảo hiệu năng xử lý ngay cả trong giờ cao điểm điểm.

Hơn nữa tính sẵn sàng là yêu cầu hệ thống cần đáp ứng. Để làm được điều đó hệ thống cần được triển khai trên cụm Kubernetes, hỗ trợ cơ chế tự phục hồi (Self-healing) và co giãn tự động (Auto-scaling) theo tải trọng thực tế của doanh nghiệp.

Ngoài ra hệ thống cần ổn định hạ tầng bằng cách sử dụng triết lý GitOps (ArgoCD) để đảm bảo cấu hình hệ thống trên môi trường Production luôn đồng nhất với mã nguồn, giảm thiểu rủi ro do lỗi cấu hình thủ công.

3.2.3. Yêu cầu bảo mật và an toàn dữ liệu

Hệ thống sử dụng chuẩn JWT (JSON Web Token) cho các phiên làm việc của người dùng và API Key mã hóa cho các kết nối hệ thống từ đó đáp ứng được yêu cầu về xác thực và ủy quyền.

Tiếp theo là an toàn dữ liệu hóa đơn. Hệ thống cần tuân thủ tuyệt đối chuẩn ký số XMLDSig, đảm bảo hóa đơn không thể bị chỉnh sửa sau khi ký. Mọi dữ liệu nhạy cảm như mật khẩu Provider hay Secret Key đều được mã hóa trước khi lưu trữ trong cơ sở dữ liệu.

Cuối cùng tính chống chối bỏ là yêu cầu hệ thống cần thực hiện. Cần lưu vết chi tiết mọi thao tác của người dùng và các giao dịch với nhà cung cấp dịch vụ hóa đơn, đảm bảo khả năng đối soát và phục hồi khi có tranh chấp dữ liệu.

IV. THIẾT KẾ HỆ THỐNG

4.1. Thiết kế tổng thể hệ thống

4.1.1. Kiến trúc phân tầng Layered Architecture

Hệ thống được thiết kế theo mô hình kiến trúc đa tầng giúp tách biệt rõ ràng các trách nhiệm và tạo điều kiện cho việc nâng cấp, kiểm thử độc lập giữa các thành phần.

Lớp trình bày Presentation Layer lớp này không chỉ đảm nhận việc hiển thị giao diện người dùng mà còn xử lý các logic trải nghiệm khách hàng, xác thực định dạng dữ liệu đầu vào Client-side validation và giao tiếp bất đồng bộ với Backend thông qua các RESTful API.

API Gateway và Lớp bảo mật Sercurity Layer sử dụng Spring Sercurity làm lá chắn trung tâm. Tại đây, hệ thống thực hiện hai luồng xác thực song song: JWT JSON Web Token dành cho các phiên làm việc của người dùng trên trình duyệt và API Key dành cho các hệ thống ERP/POS bên ngoài tích hợp tự động. Lớp này đảm bảo tính toàn vẹn và ngăn chặn các truy cập trái phép vào vùng nghiệp vụ nhạy cảm.

Bussiness Logic Layer hay lớp dịch vụ Service Layer đây là nơi xử lý chính của hệ thống, nơi thực thi các quy trình cốt lõi như quản lý vòng đời hóa đơn, kiểm tra hạn mức Quota và điều phối các giao dịch tài chính. Đặc biệt, lớp này tích hợp Java 21 Virtual Threads, cho phép xử lý hàng ngàn yêu cầu nghiệp vụ đồng thời mà không bị giới hạn bởi số lượng Thread vật lý của máy chủ.

Lớp cầu nối Adapter Layer là thành phần đặc thù của mô hình Hub. lớp này có nhiệm vụ chuyển đổi Mapping các đối tượng dữ liệu chuẩn của Hub sang định dạng XML/JSON cụ thể mà mỗi nhà cung cấp MISA, BKAV, VNPT... yêu cầu. Việc tách biệt lớp này giúp logic lõi của hệ thống không bị phụ thuộc vào sự thay đổi API của bên thứ ba.

Cuối cùng là lớp lưu trữ Persistence Layer sử dụng Spring Data JPA để giao tiếp với MariaDB. Hệ thống áp dụng chiến lược lưu trữ tập trung, quản lý đồng thời dữ liệu cấu trúc và dữ liệu Payload lớn trong một đơn vị giao dịch duy nhất, đảm bảo tính nhất quán (ACID) tuyệt đối cho các nghiệp vụ thuế.

4.1.2. Mô hình Middleware Hub và Adapter Pattern

Hệ thống EInvoiceHub vận hành dựa trên triết lý đồng nhất trong đa dạng. Để hiện thực hóa điều này, dự án ứng dụng kết hợp hai mẫu thiết kế kinh điển là Strategy Pattern và Adapter Pattern.Cơ chế trừu tượng hóa qua việc hệ thống định nghĩa một giao diện chung duy nhất là InvoiceProvider. Mọi hành động như phát hành, ký số, hay tra cứu đều được trừu tượng hóa thành các phương thức tiêu chuẩn. Điều này giúp lớp Service chỉ cần tương tác với Interface mà không cần quan tâm hóa đơn đó sẽ được gửi tới nhà cung cấp nào.

Mẫu thiết kế Adapter được hiểu là mỗi nhà cung cấp dịch vụ hóa đơn TVAN sẽ có một Adapter riêng (ví dụ: MisaAdapter, BkavAdapter). Các Adapter này thực hiện nhiệm vụ đóng gói sự phức tạp trong quy trình

gọi API, xử lý mã lỗi và định dạng chữ ký số đặc thù của từng nhà cung cấp. Khi có một nhà cung cấp mới, lập trình viên chỉ cần tạo thêm một Adapter mới tuân thủ giao diện InvoiceProvider mà không phải sửa đổi bất kỳ dòng mã nguồn nào trong logic phát hành hóa đơn chính.

Điều phối linh hoạt Strategy & Factory Pattern thông qua ProviderFactory, hệ thống sẽ dựa trên cấu hình của từng doanh nghiệp để quyết định chiến lược (Strategy) xử lý phù hợp tại thời điểm thực thi. Điều này tạo ra khả năng tùy biến cực cao cùng một hệ thống Hub nhưng Doanh nghiệp A có thể xuất hóa đơn qua MISA, trong khi Doanh nghiệp B lại sử dụng BKAV một cách hoàn toàn minh bạch.

EInvoiceHub không chỉ là điểm chuyển tiếp dữ liệu, mà còn đóng vai trò là lớp Buffer và Validator vừa là vùng đệm vừa là nơi xác thực . Hệ thống thực hiện kiểm tra tính hợp lệ pháp lý của hóa đơn trước khi gửi sang Provider, đồng thời lưu trữ lại toàn bộ "vết" giao dịch và Payload gốc. Điều này giúp doanh nghiệp chủ động hoàn toàn trong việc đối soát dữ liệu mà không bị lệ thuộc vào hạ tầng lưu trữ của các nhà cung cấp bên thứ ba.

4.2. Thiết kế cơ sở dữ liệu

4.2.1. Chiến lược lưu trữ MariaDB tập trung

Trong quá trình phát triển, thay vì sử dụng mô hình lưu trữ đa nền tảng dùng cả SQL và NoSQL, hệ thống đã được chuẩn hóa để vận hành trên một hệ quản trị cơ sở dữ liệu quan hệ duy nhất là MariaDB. Chiến lược này mang lại tính nhất quán cao và đơn giản hóa quy trình vận hành trên hạ tầng Kubernetes.

Hóa đơn điện tử là dữ liệu tài chính nhạy cảm, đòi hỏi sự chính xác tuyệt đối. Việc sử dụng MariaDB đảm bảo tính nhất quán và toàn vẹn giữa thông tin trạng thái hóa đơn và nội dung thực tế trong cùng một đơn vị giao dịch, tránh rủi ro mất dữ liệu XML khi nghiệp vụ đã ghi nhận thành công.

Hệ thống phân tách dữ liệu thành hai nhóm logic nhưng lưu trữ trong cùng một hạ tầng để tối ưu hóa việc quản lý. Dữ liệu nghiệp vụ được hệ thống lưu trữ như các thông tin định danh, số hóa đơn, mã số thuế, và các chỉ số tài chính. Nhóm này được thiết kế chuẩn hóa để phục vụ các truy vấn phức tạp, bộ lọc dữ liệu và kết xuất báo cáo tốc độ cao. Dữ liệu nội dung như các tệp XML hóa đơn gốc và tệp đã ký có kích thước lớn được lưu trữ dưới dạng LONGTEXT hoặc JSON. Hệ thống ứng dụng kỹ thuật Lazy Loading - chỉ tải nội dung khi thực sự cần thiết để giảm tải cho bộ nhớ RAM và tối ưu hóa băng thông truyền tải.

Đề tài nghiên cứu ứng dụng kỹ thuật Indexing trên các trường định danh mã tra cứu và Partitioning phân vùng dữ liệu theo thời gian để duy

trì tốc độ phản hồi dưới 1 giây ngay cả khi quy mô dữ liệu đạt mức hàng triệu bản ghi.

4.2.2. Sơ đồ thực thể quan hệ (ERD)

Sơ đồ ERD – đang hoàn thiện

4.3. Thiết kế giao diện

Giao diện hệ thống được phát triển trên nền tảng Angular, tập trung vào tiêu chuẩn UI/UX hiện đại, đảm bảo sự chuyên nghiệp và tính chính xác cao cho nghiệp vụ quản lý tài chính.

Dashboard điều khiển trung tâm Management Dashboard cung cấp cái nhìn tổng thể cho người quản trị thông qua các biểu đồ trực quan. Người dùng có thể theo dõi tức thời biến động của hạn mức hóa đơn Quota usage, thống kê tỷ lệ hóa đơn thành công/thất bại và các cảnh báo hệ thống từ phía nhà cung cấp.

Quản lý hóa đơn tập trung qua thiết kế dạng bảng lưới Data Grid mạnh mẽ, hỗ trợ bộ lọc đa điều kiện theo ngày, theo khách hàng, theo trạng thái. Tích hợp chức năng xem nhanh Quick View nội dung hóa đơn và hỗ trợ tải xuống hàng loạt tệp XML/PDF chỉ với một thao tác.

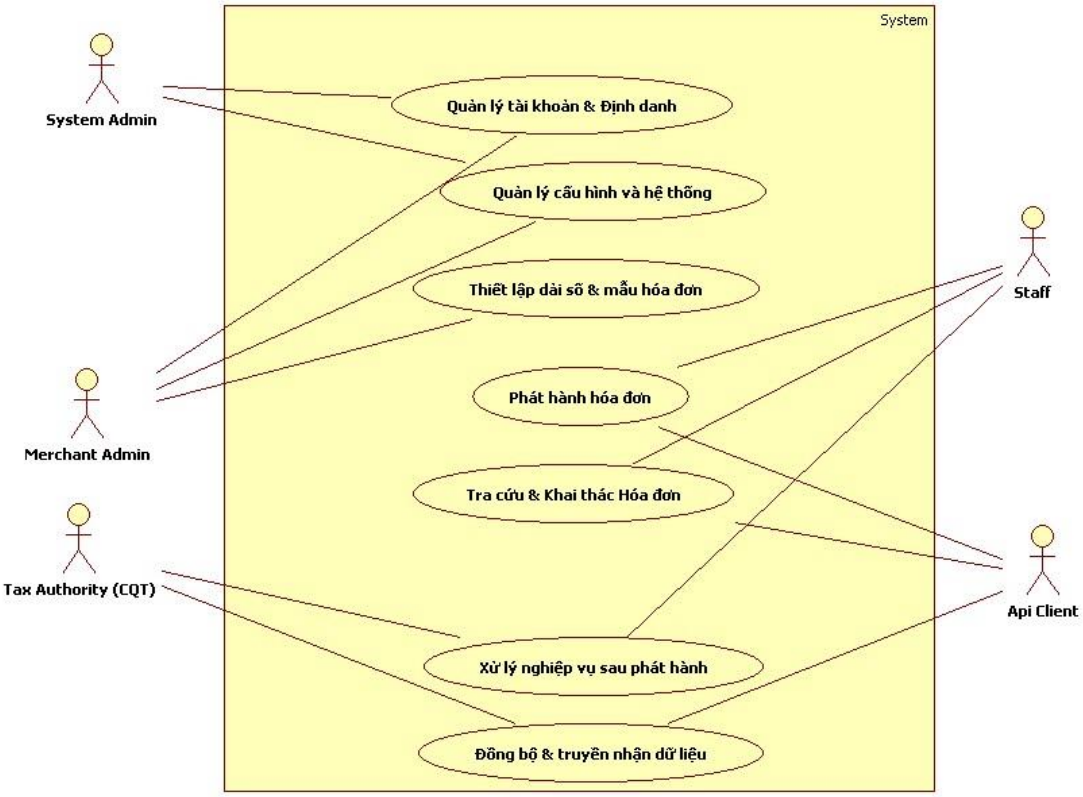
Cấu hình kết nối linh hoạt bằng giao diện hướng dẫn từng bước Wizard giúp Merchant Admin dễ dàng thiết lập thông số kết nối Provider. Tích hợp công cụ kiểm tra kết nối thời gian thực để đảm bảo cấu hình chính xác trước khi đưa vào vận hành.

Tính tương thích và Responsive Design giao diện được xây dựng dựa trên kiến trúc Grid linh hoạt, đảm bảo hiển thị tối ưu trên đa dạng thiết bị từ máy tính để bàn đến các thiết bị di động, giúp nhân viên kế toán có thể giám sát trạng thái hóa đơn mọi lúc mọi nơi.

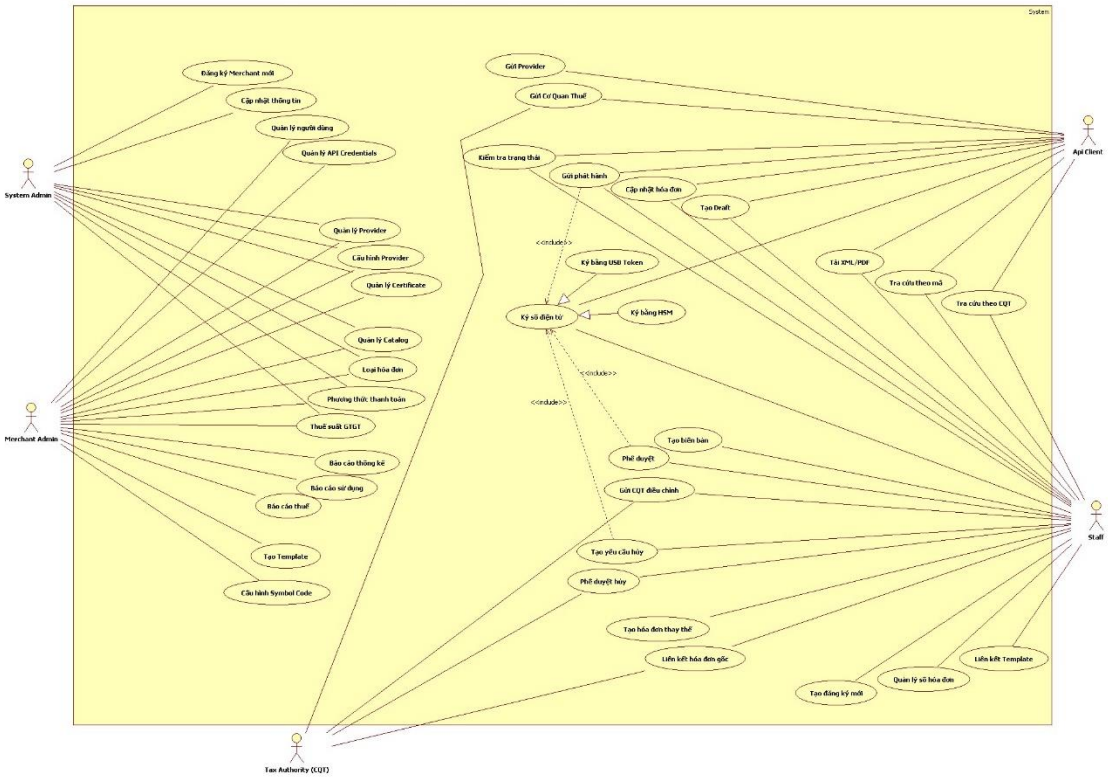
4.4. Thiết kế hệ thống UML

4.4.1. Sơ đồ Trường hợp sử dụng - Use Case Diagram

Hệ thống được thiết kế để quản lý toàn bộ vòng đời của hóa đơn, từ khâu đăng ký dải số, phát hành, ký số đến các nghiệp vụ sau phát hành như điều chỉnh, thay thế và hủy bỏ.



Hình 3: Sơ đồ Trường hợp sử dụng tổng quát



Hình 4: Sơ đồ trường hợp sử dụng chi tiết

4.4.1.1. Các tác nhân – Actor

Hệ thống xác định 05 tác nhân chính, đại diện cho các thực thể người dùng và các hệ thống bên thứ ba tương tác trực tiếp với Hub:.

System Admin là tác nhân quản trị hệ thống Hubcó đặc quyền cao nhất, chịu trách nhiệm vận hành hạ tầng kỹ thuật, quản lý các thực thể Merchant, thiết lập kết nối với các đối tác Provider và duy trì các danh mục chuẩn quốc gia.

Merchant Admin là tác nhân quản trị doanh nghiệp đại diện cho cấp quản lý tại mỗi đơn vị kinh doanh. Tác nhân này chịu trách nhiệm cấu hình các tham số nghiệp vụ riêng biệt của doanh nghiệp mình, quản lý đội ngũ nhân viên (Staff) và thiết lập các mã định danh API (API Credentials).

Staff là Nhân viên nghiệp vụ hoặc Kế toán đây là các tác nhân trực tiếp tương tác với giao diện người dùng để thực hiện các thao tác thủ công hàng ngày như lập hóa đơn, ký số, tra cứu dữ liệu và xử lý các hóa đơn sai sót.

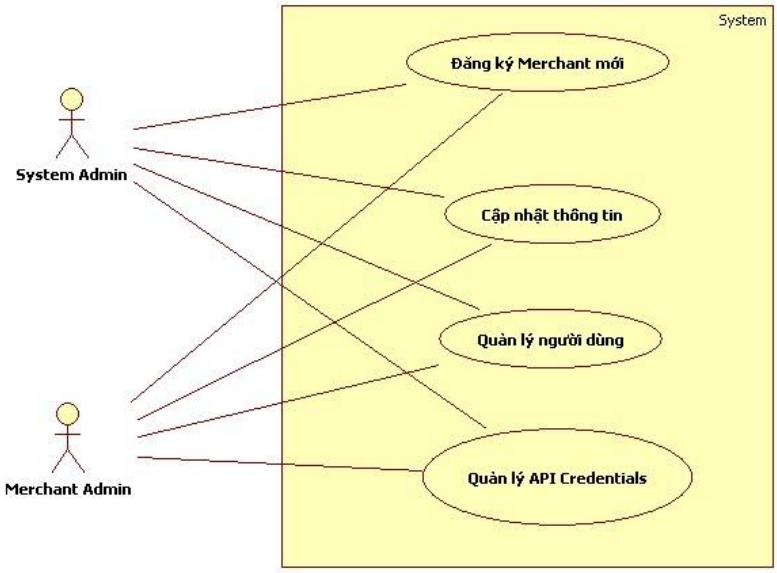
API Client hay Hệ thống tích hợp là một tác nhân hệ thống đại diện cho các phần mềm ERP, POS hoặc CRM. Tác nhân này tương tác với Hub qua giao thức REST API để tự động hóa quy trình phát hành hóa đơn từ dữ liệu bán hàng sẵn có..

Tax Authority (Cơ quan Thuế) đây là tác nhân hệ thống đóng vai trò kiểm soát tính pháp lý. Hub tương tác với tác nhân này thông qua Provider để nhận mã cấp hóa đơn và phản hồi trạng thái phê duyệt hóa đơn..

4.4.1.2. Các nhóm trường hợp sử dụng – Use Case

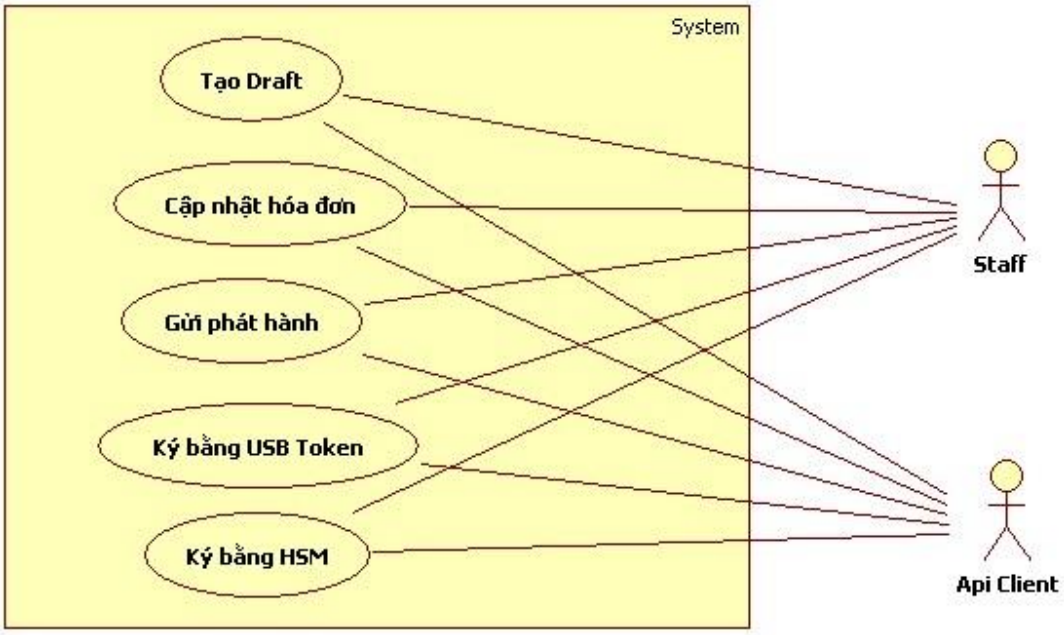
Các chức năng được phân nhóm theo các miền nghiệp vụ logic để đảm bảo tính dễ hiểu và quản trị.

Nhóm Quản lý định danh Merchant: Tập trung vào việc thiết lập thực thể kinh doanh và quyền kiểm soát truy cập. Trong đó đăng ký và cập nhật Merchant cho phép đưa doanh nghiệp mới vào hệ thống, duy trì thông tin pháp lý (Mã số thuế, địa chỉ, người đại diện) đảm bảo tính chính xác của hóa đơn xuất ra. Quản lý người dùng nội bộ giúp Merchant Admin thực hiện phân quyền chi tiết cho nhân viên Staff theo từng phòng ban hoặc chi nhánh. Quản lý API Credentials sử dụng để thiết lập và cấp phát cặp Client ID/API Key để xác thực các kết nối tự động từ hệ thống vệ tinh, đảm bảo an toàn dữ liệu trên môi trường mạng.

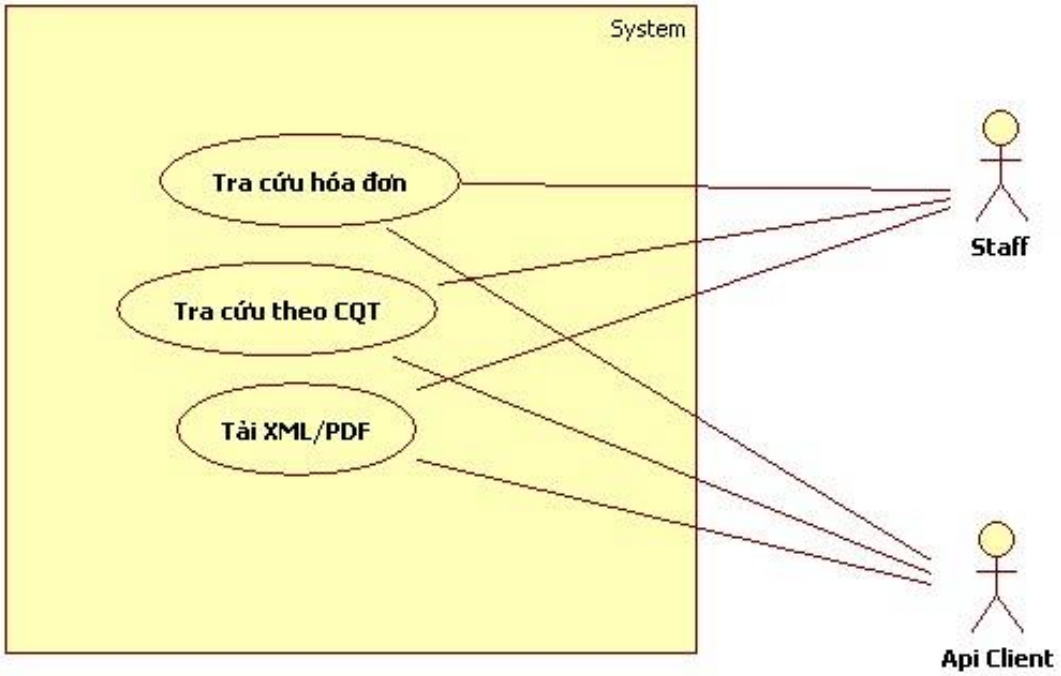


Hình 5: Sơ đồ Trường hợp sử dụng phân rã Use Case quản lý tài khoản định danh và quản lý cấu hình hệ thống

Nhóm Vận hành hóa đơn (Core Operations): Trọng tâm xử lý kỹ thuật và nghiệp vụ tài chính. Trong đó khởi tạo hóa đơn (Drafting) cho phép tiếp nhận dữ liệu, kiểm chuẩn định dạng và tạo hóa đơn nháp trước khi phát hành chính thức. Hệ thống hỗ trợ đa phương thức ký số điện tử tự động qua sử dụng Cloud HSM để xử lý hóa đơn lô lớn với tốc độ cao. Ký thủ công qua tích hợp qua USB Token dành cho các giao dịch đơn lẻ. Đồng bộ hóa dữ liệu Provider/CQT giúp tự động điều phối việc chuyển dữ liệu XML đã ký sang các nhà cung cấp dịch vụ và theo dõi phản hồi từ Cơ quan Thuế.Tra cứu và khai thác dữ liệu cung cấp bộ lọc tìm kiếm thông minh, cho phép truy xuất Payload XML gốc và bản thể hiện PDF để gửi cho khách hàng.



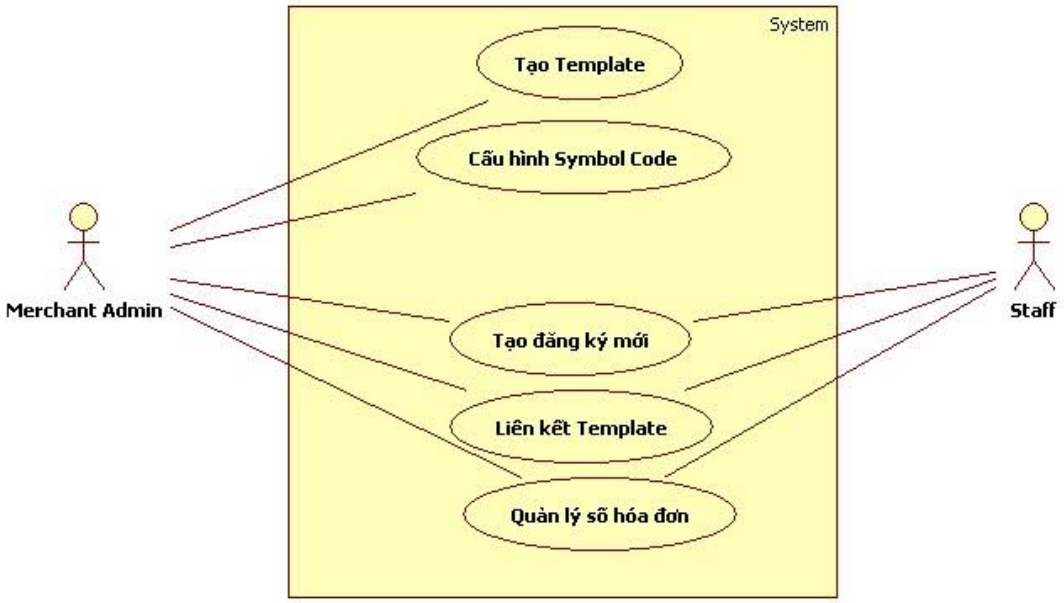
***Hình 6*: Sơ đồ Trường hợp sử dụng phân rã Use Case phát hành hóa đơn**



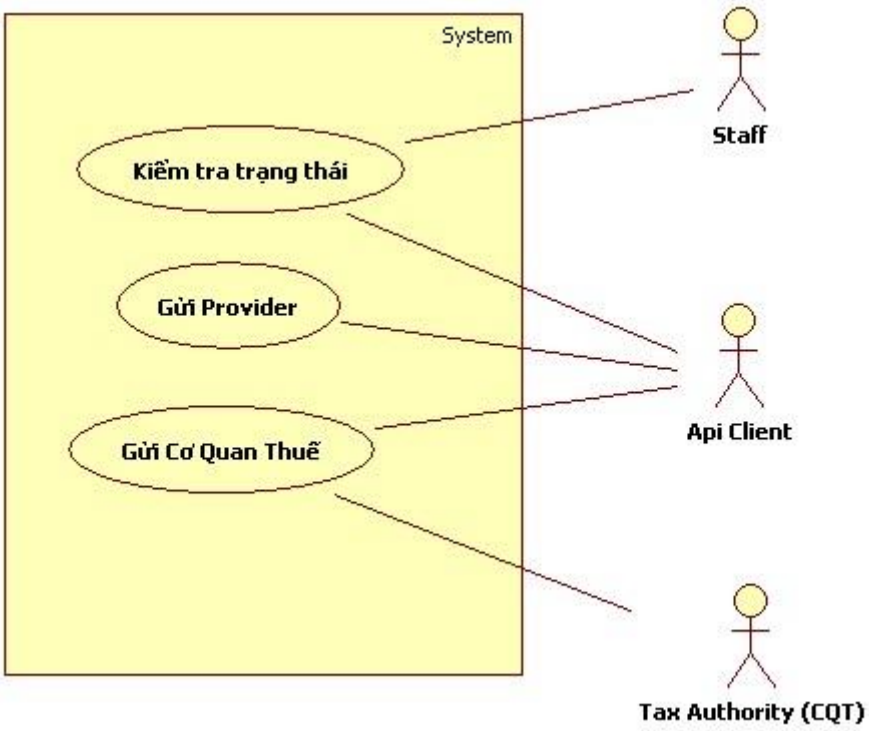
Hình 7: Sơ đồ Trường hợp sử dụng phân rã Use Case phát hành hóa đơn và tra cứu và khai thác hóa đơn

Nhóm Quản lý đăng ký pháp lý: Đảm bảo tính tuân thủ quy định của Bộ Tài chính. Đăng ký dải số hóa đơn cho phép quản lý số lượng và khoảng số đã được CQT phê duyệt, thực hiện cảnh báo khi hạn mức sắp hết. Thiết kế mẫu và ký hiệu qua Cấu hình các mẫu hóa đơn theo Thông tư 78, đảm bảo mọi hóa đơn xuất ra đều đạt chuẩn về hiển thị và pháp lý.

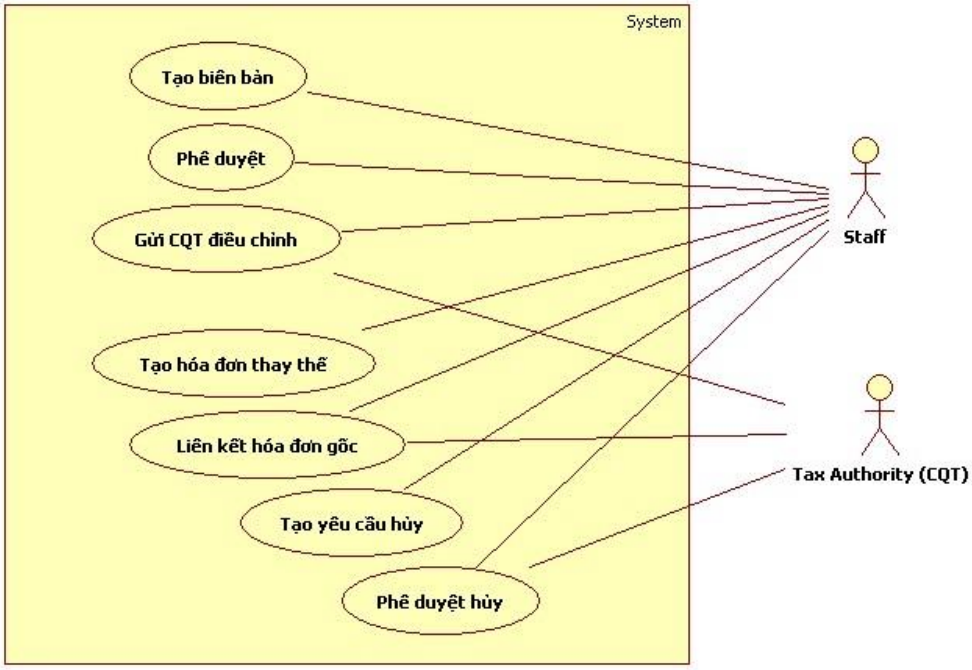
Nhóm Nghiệp vụ điều chỉnh sau phát hành: Xử lý các tình huống phát sinh và sai sót dữ liệu. Cho phép điều chỉnh hóa đơn qua việc lập biên bản thỏa thuận điện tử và phát hành hóa đơn điều chỉnh giá trị hoặc thông tin định danh. Có thể thay thế hóa đơn bằng quy trình hủy hóa đơn lỗi và phát hành hóa đơn mới có liên kết tham chiếu tới bản gốc. Hỗ trợ hủy hóa đơn bằng cách thực hiện xóa bỏ giá trị pháp lý của hóa đơn trên hệ thống của CQT khi có yêu cầu phê duyệt hợp lệ.



Hình 8: Sơ đồ Trường hợp sử dụng phân rã Use Case quản lý tài khoản định danh và thiết lập dải số và mẫu hóa đơn



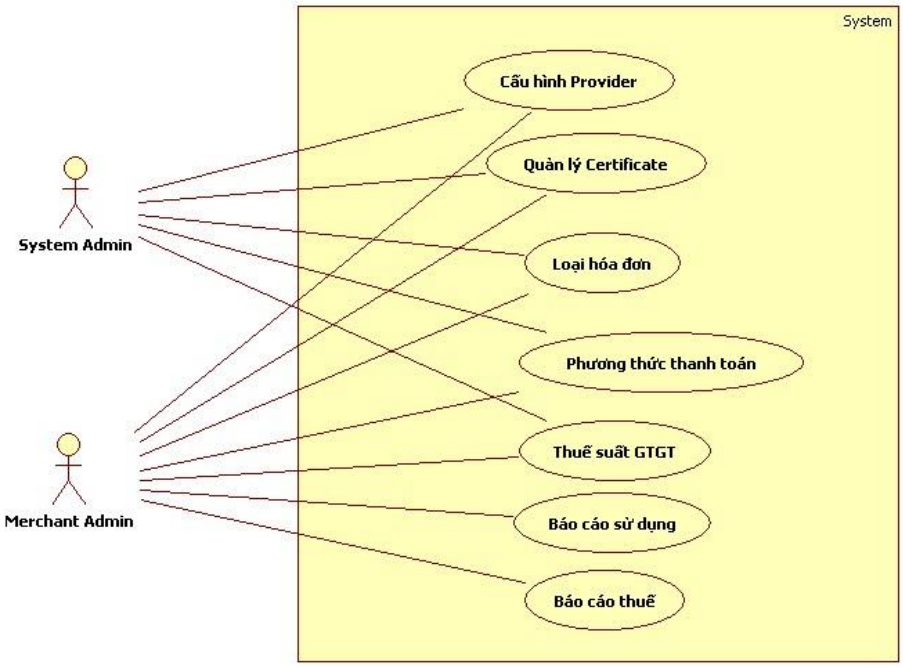
Hình 9: Sơ đồ Trường hợp sử dụng phân rã Use Case phát hành hóa đơn



Hình 10: Sơ đồ Trường hợp sử dụng phân rã Use Case xử lý nghiệp vụ sau phát hành

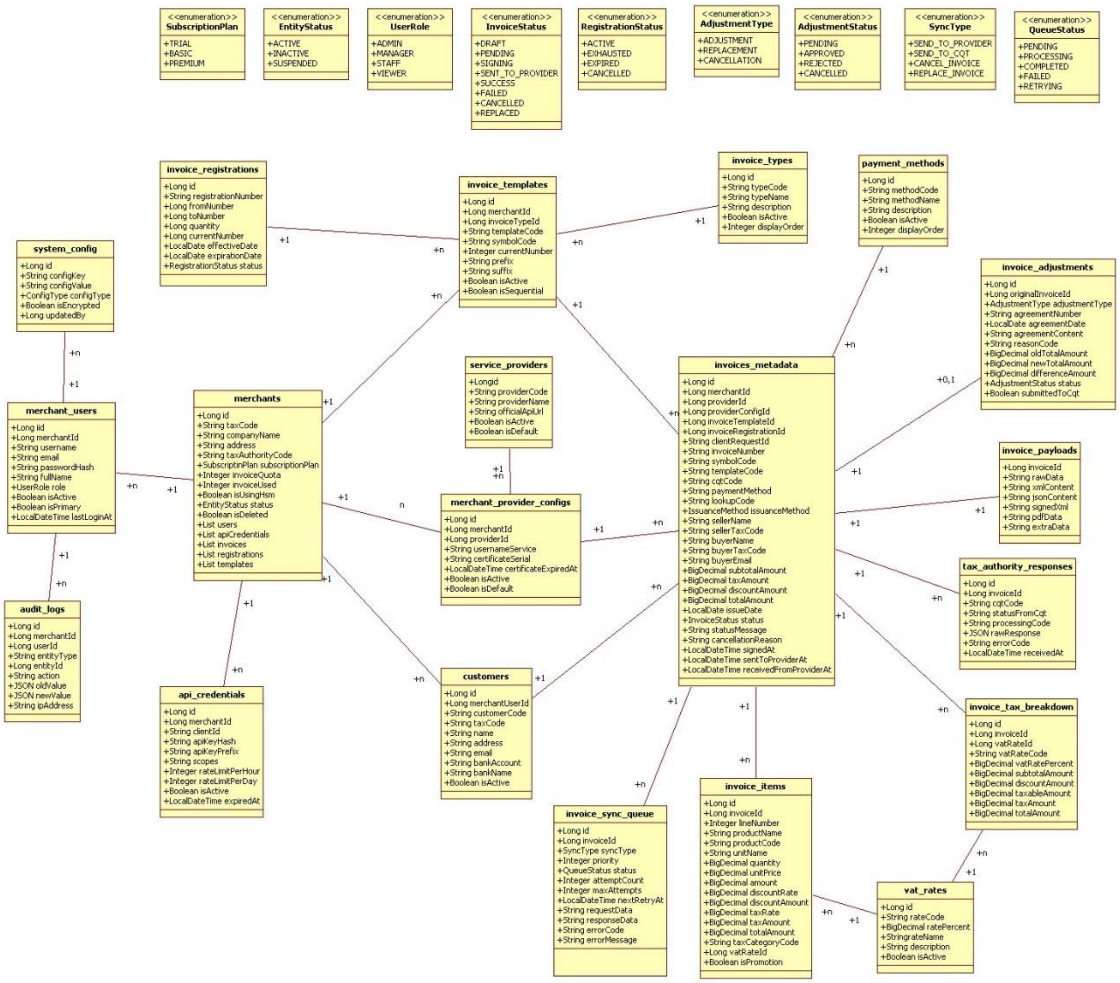
Nhóm Quản trị hệ thống và Báo cáo: Dành cho công tác vận hành và giám sát: Hệ thống có chức năng cấu hình Provider & Certificate cho phép quản lý tham số kết nối API tới các TVAN và giám sát thời hạn của

các chứng thư số hệ thống. Quản lý Catalog chuẩn từ việc thiết lập các danh mục dùng chung như Loại hóa đơn, Thuế suất GTGT, Đơn vị tính để đồng bộ dữ liệu trên toàn hệ thống.Chức năng báo cáo thống kê để cung cấp dữ liệu tổng hợp về tình hình phát hành, tình trạng sử dụng hạn mức và hỗ trợ kết xuất báo cáo thuế định kỳ cho doanh nghiệp.



Hình 11: Sơ đồ Trường hợp sử dụng phân rã Use Case quản lý cấu hình và hệ thống

4.4.2. Sơ đồ Lớp – Class Diagram



Hình 12: Sơ đồ Lớp tổng quát

Hình 13: Sơ đồ Lớp tổng quát

4.4.2.1. Các nhóm thực thể chính

Để tối ưu hóa việc quản lý vận hành, các lớp trong hệ thống được chia thành 4 nhóm logic chính

Nhóm quản lý doanh nghiệp đóng vai trò thiết lập định danh và cơ chế bảo mật cho từng khách hàng tổ chức. Trong đó Merchants là lớp thực thể trung tâm lưu trữ thông tin pháp lý (Mã số thuế, đại diện pháp luật). Thực thể này quản lý các tham số vận hành quan trọng như hạn mức sử dụng và trạng thái hoạt động của doanh nghiệp. Merchant_users quản lý tài khoản người dùng thuộc phạm vi doanh nghiệp. Sử dụng thuộc tính UserRole để phân cấp quyền hạn (ADMIN, STAFF, VIEWER) trong nội bộ Merchant. Api_credentials cung cấp cơ chế xác thực cho các hệ thống ngoại vi (ERP/POS). Lưu trữ client_id và api_key_hash giúp bảo mật các kết nối tự động thông qua giao thức REST API.

Nhóm nghiệp vụ hóa đơn là nhóm thực thể có mật độ giao dịch cao nhất, lưu trữ chi tiết vòng đời của từng hóa đơn. Trong đó Invoices_metadata lưu trữ thông tin tóm tắt, trạng thái xử lý (DRAFT, PENDING, SUCCESS...) và các thông tin định danh của người mua/người bán. Lớp Invoice_items chứa thông tin chi tiết danh mục

hàng hóa, dịch vụ trong hóa đơn. Mỗi dòng bao gồm đơn giá, số lượng và mức thuế suất riêng biệt, cho phép tái hiện hóa đơn một cách chính xác. Lớp Invoice_tax_breakdown thực hiện phân rã và tổng hợp tiền thuế theo từng nhóm thuế suất, phục vụ trực tiếp cho việc trích xuất báo cáo thuế định kỳ. Lớp Invoice_payloads sử dụng chiến lược lưu trữ nội dung thô (XML/JSON) và tệp tin đã ký số (signed_xml) dưới dạng LONGTEXT, đảm bảo tính toàn vẹn dữ liệu cho mục đích tra cứu và tải về bản thể hiện PDF.

Nhóm Tuân thủ và Đồng bộ đảm bảo tính sẵn sàng và tính pháp lý của dữ liệu khi tương tác với bên thứ ba. Trong đó Invoice_registrations quản lý các dải số hóa đơn (Pattern, Serial) đã được Cơ quan Thuế chấp nhận cho phép Merchant sử dụng. Lớp Invoice_sync_queue là thành phần quan trọng trong kiến trúc hệ thống, quản lý hàng đợi gửi dữ liệu sang Provider. Hỗ trợ cơ chế Retry tự động thông qua attempt_count để xử lý các sự cố mạng tạm thời. Lớp tax_authority_responses giúp lưu trữ các phản hồi chính thức và mã cấp từ Cơ quan Thuế , là bằng chứng pháp lý cao nhất của một hóa đơn điện tử. Lớp Audit_logs hỗ trợ ghi lại chi tiết mọi biến động dữ liệu (ai thay đổi, thay đổi nội dung gì, vào lúc nào), đảm bảo khả năng truy vết và tính minh bạch của hệ thống.

4.4.2.2. Các mối quan hệ

Quan hệ sở hữu và Phân cấp quản lý. Trong đó Merchant (1:N) Merchant_users & Api_credentials đây là mối quan hệ cốt lõi thể hiện tính Multi-tenancy. Một doanh nghiệp (Merchant) sở hữu nhiều tài khoản nhân viên và nhiều cặp định danh API. Ràng buộc khóa ngoại merchant_id đảm bảo dữ liệu của Merchant này hoàn toàn tách biệt với Merchant khác. Merchant (1:N) Invoice_templates & Customers doanh nghiệp chủ động quản lý các mẫu hóa đơn riêng và danh sách khách hàng thường xuyên của mình, tạo nên một hệ sinh thái dữ liệu độc lập.

Quan hệ thành phần nghiệp vụ hóa đơn. Trong đó Invoices_metadata (1:N) Invoice_items một hóa đơn tổng thể bao gồm nhiều dòng hàng hóa chi tiết. Mối quan hệ này sử dụng ràng buộc ON DELETE CASCADE, nghĩa là khi một bản ghi metadata bị xóa (hoặc rollback), các item liên quan sẽ tự động được xử lý để tránh rác dữ liệu. Invoices_metadata (1:1) Invoice_payloads đây là mối quan hệ ánh xạ 1-1 đặc biệt. Metadata lưu trữ thông tin tra cứu nhanh, trong khi Payload lưu trữ dữ liệu thô (XML/JSON) dung lượng lớn. Việc tách biệt này giúp tối ưu hóa bộ nhớ đệm (Cache) và tốc độ truy vấn danh sách.

Quan hệ tham chiếu danh mục chuẩn. Trong đó Invoices_metadata (N:1) Invoice_types, Vat_rates & Payment_methods là các thực thể nghiệp vụ tham chiếu tới các bảng danh mục chuẩn để đảm bảo dữ liệu

luôn tuân thủ các quy định của Tổng cục Thuế (ví dụ: mã loại hóa đơn 01GTKT, thuế suất 8%). Việc dùng khóa ngoại tới danh mục giúp ngăn chặn các sai lệch định dạng khi truyền nhận API với các Provider.

Quan hệ giám sát và Đồng bộ. Trong đó Invoices_metadata (1:N) Provider_transactions và Tax_authority_responses giúp lưu vết mọi tương tác giữa Hub với hệ thống bên ngoài. Mỗi hóa đơn có thể có nhiều phiên giao dịch (nếu có lỗi và phải gửi lại), giúp quản trị viên hệ thống có thể đối soát và xử lý sự cố kịp thời..

4.4.2.3. Các ràng buộc và đặc tính kỹ thuật

Ràng buộc toàn vẹn và Nhất quán dữ liệu. Khóa ngoại (Foreign Keys) cho thất mọi mối quan hệ đều được thực thi ở mức cơ sở dữ liệu (CONSTRAINT fk_...). Điều này đảm bảo không có bản ghi rời rạc.

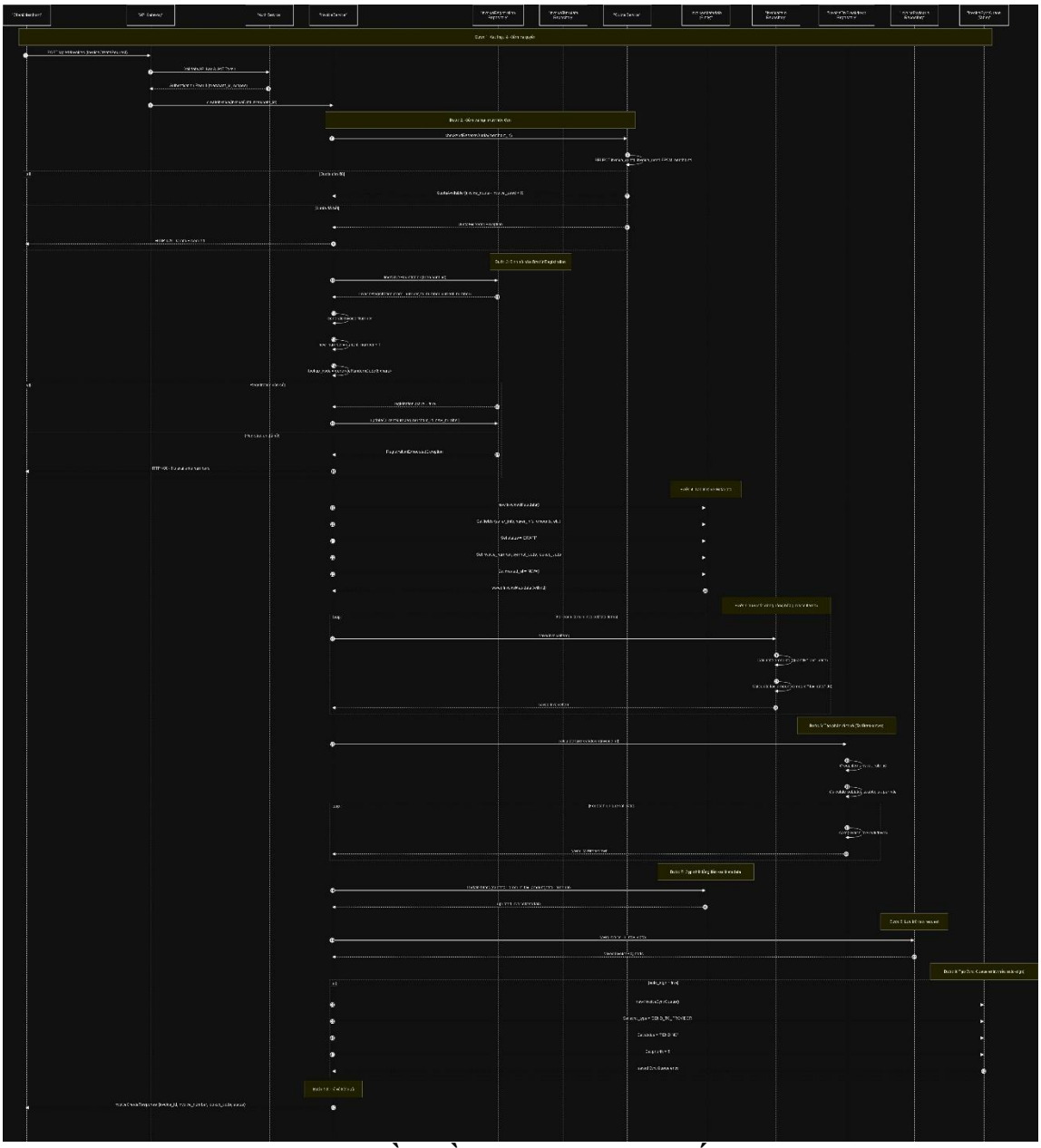
Ràng buộc duy nhất (unique): *uq_merchant_provider: Đảm bảo mỗi doanh nghiệp chỉ có một cấu hình duy nhất cho mỗi nhà cung cấp. uq_invoice_lookup_code có nghĩa mã tra cứu hóa đơn là duy nhất trên toàn hệ thống để đảm bảo khách hàng lẻ luôn tìm đúng hóa đơn của mình.

Ràng buộc kiểm tra (check constraints) sử dụng chk_vat_rate_percent và chk_symbol_code_format để kiểm soát định dạng dữ liệu ngay từ tầng lưu trữ, ngăn chặn dữ liệu rác xâm nhập.

4.4.3. Sơ đồ Tuần tự - Sequences Diagram

4.4.3.1. Quy trình tạo hóa đơn

Quy trình này mô tả các bước xử lý nội bộ của hệ thống EinvoiceHub từ khi nhận được yêu cầu xuất hóa đơn từ phía khách hàng (Merchant) cho đến khi hóa đơn được khởi tạo thành công trong cơ sở dữ liệu và sẵn sàng để ký số/gửi nhà cung cấp. Quy trình đảm bảo tính toàn vẹn dữ liệu, kiểm tra hạn mức và tuân thủ các quy định về đánh số hóa đơn.



Hình 14: Sơ đồ Tuần tự quy trình xuất hóa đơn

Các thành phần tham gia

Bảng 1: Bảng các thành phần tham gia quy trình xuất hóa đơn

Tên thành phần	Ý nghĩa
Client/Merchant	Hệ thống hoặc người dùng gửi yêu cầu tạo hóa đơn.
API Gateway	Cổng tiếp nhận, điều phối request và giới hạn lưu lượng.
Auth Service	Dịch vụ xác thực và phân quyền (xử lý API Key/JWT).
InvoiceService	Trung tâm điều phối logic nghiệp vụ tạo hóa đơn.
Repositories (RegRepo, TemplateRepo, ItemRepo, TaxRepo, PayloadRepo)	Các lớp truy xuất dữ liệu MariaDB để lưu trữ metadata, chi tiết hàng hóa, dải số và nội dung thô.
Quota Service	Dịch vụ kiểm soát hạn mức hóa đơn của doanh nghiệp.
InvoicesMetadata	Thực thể lưu trữ thông tin bao quát của hóa đơn.
InvoiceSyncQueue	Bảng hàng chờ để xử lý đồng bộ hóa đơn lên nhà cung cấp (MISA, BKAV,...) một cách bất đồng bộ.

Chi tiết quy trình.

Bước 1: Xác thực & Kiểm tra quyền (Authentication). Khách hàng gửi yêu cầu POST chứa dữ liệu hóa đơn đến API Gateway. Gateway gọi Auth Service để kiểm tra tính hợp lệ của API Key hoặc Token. Nếu hợp lệ, Auth Service trả về thông tin merchant_id và các quyền (scopes) tương ứng. Yêu cầu sau đó được chuyển đến InvoiceService.

Bước 2: Kiểm tra hạn mức (Quota Check). InvoiceService gọi Quota Service để kiểm tra xem doanh nghiệp còn đủ số lượng hóa đơn để xuất hay không. Logic là hệ thống truy vấn bảng merchants để so sánh invoice_quota (hạn mức) và invoice_used (đã dùng).Xử lý ngoại lệ nếu hết hạn mức, hệ thống ném ra QuotaExceededException và trả về lỗi HTTP 429 cho khách hàng.

Bước 3: Sinh số hóa đơn & Mã tra cứu (Number Generation). Hệ thống tìm kiếm dải hóa đơn đang hoạt động (Registration) của doanh nghiệp. Logic sinh số lấy current_number hiện tại cộng thêm 1 để làm

số hóa đơn mới. Đồng thời, sinh một lookup_code (mã tra cứu) ngẫu nhiên gồm 6 ký tự để phục vụ việc tra cứu hóa đơn sau này. Xử lý ngoại lệ nếu dải số đã hết (vượt quá to_number), hệ thống trả về lỗi "No available numbers".

Bước 4: Khởi tạo thông tin chung (Invoice Metadata).Khởi tạo thực thể InvoicesMetadata để lưu các thông tin "vỏ" của hóa đơn như: Thông tin người mua, người bán, ngày lập, trạng thái mặc định là DRAFT (Nháp) và số hóa đơn vừa sinh.

Bước 5: Lưu chi tiết hàng hóa (Invoice Items).Hệ thống lặp qua danh sách hàng hóa khách hàng gửi lên. Với mỗi mặt hàng, hệ thống tự động tính toán: thành tiền = số lượng * đơn giá và tiền thuế = thành tiền * thuế suất.Dữ liệu được lưu vào bảng invoice_items liên kết với ID của hóa đơn mẹ.

Bước 6: Phân rã thuế (Tax Breakdown). Để đảm bảo tính chính xác cho báo cáo thuế, hệ thống thực hiện nhóm các mặt hàng theo từng loại thuế suất (ví dụ: nhóm 5%, nhóm 10%). Tính toán tổng tiền chưa thuế và tổng tiền thuế cho từng nhóm rồi lưu vào bảng InvoiceTaxBreakdown.

Bước 7: Cập nhật tổng tiền vào Metadata. Sau khi tính toán xong chi tiết, InvoiceService cập nhật lại các trường tổng cộng (subtotal_amount, tax_amount, total_amount) vào bản ghi Metadata đã tạo ở Bước 4 để đảm bảo dữ liệu thống nhất.

Bước 8: Lưu trữ dữ liệu thô (Payload Storage). Toàn bộ nội dung yêu cầu (JSON/XML) được lưu vào bảng invoice_payloads. Việc này giúp hệ thống có thể tái tạo lại hóa đơn gốc hoặc phục vụ mục đích kiểm soát (Audit) mà không làm ảnh hưởng đến hiệu năng của bảng Metadata.

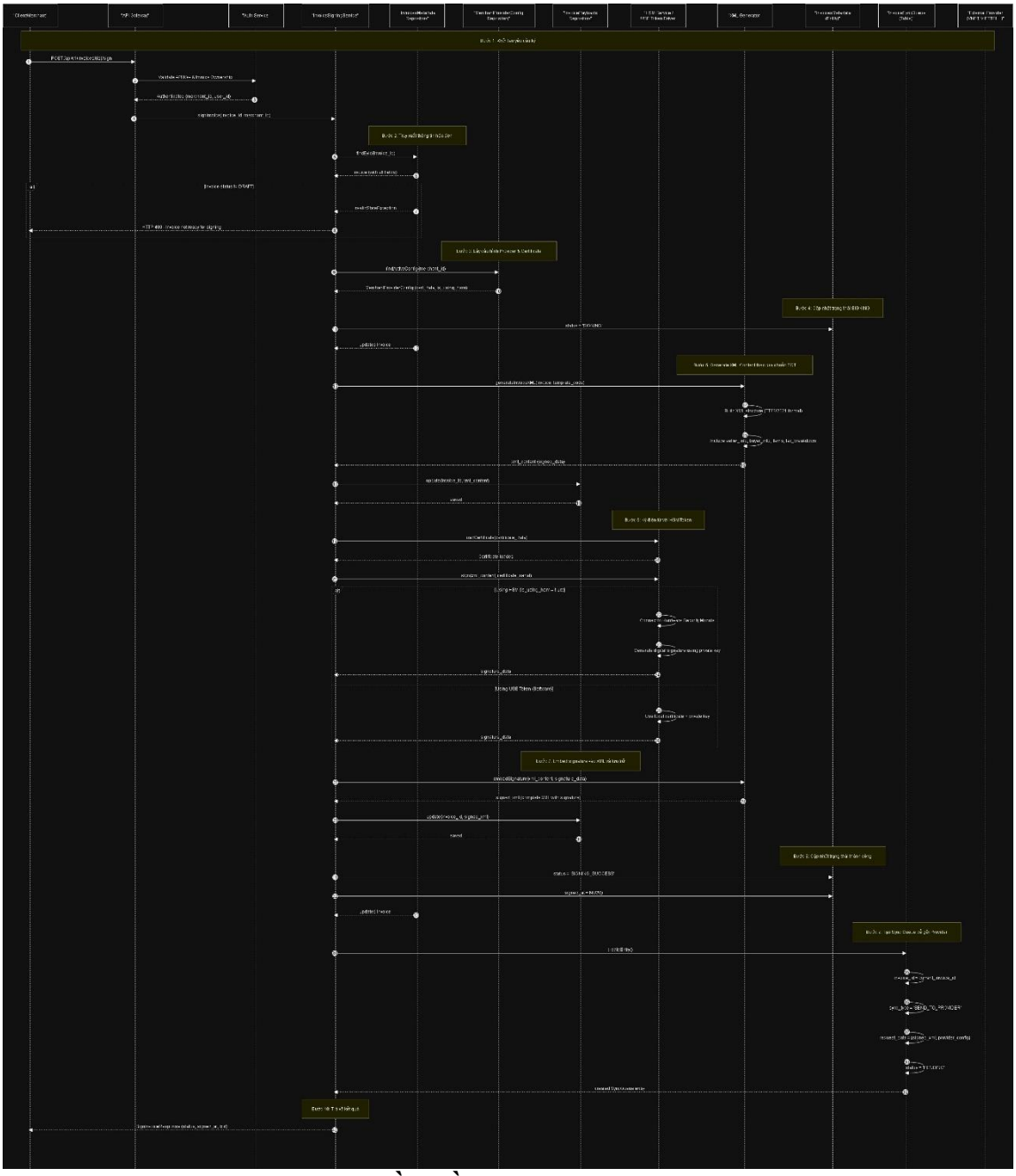
Bước 9: Tạo hàng chờ đồng bộ (Sync Queue).Nếu Merchant cấu hình chế độ Tự động ký/Tự động gửi (auto_sign = true), InvoiceService sẽ tạo một bản ghi trong hàng chờ InvoiceSyncQueue.Trạng thái được đặt là PENDING để các Worker (Background Tasks) sau đó sẽ lấy ra và thực hiện gọi API sang các nhà cung cấp như MISA, BKAV hoặc Viettel.

Bước 10: Trả về kết quả sau khi hoàn tất các bước lưu trữ nội bộ, hệ thống trả về cho Client phản hồi thành công bao gồm: invoice_id, invoice_number, lookup_code và trạng thái hiện tại của hóa đơn

Các điểm thiết kế đáng kể là tính nhất quán (Consistency) qua quy trình sử dụng các bước tính toán lại tổng tiền từ Item trước khi cập nhật Metadata để tránh sai lệch dữ liệu giữa các bảng.Tính mở rộng trong việc sử dụng SyncQueue (Bước 9) cho phép hệ thống phản hồi nhanh cho người dùng mà không cần chờ kết quả từ bên thứ 3 (Provider), giúp hệ thống chịu tải tốt hơn. Bảo mật khi kiểm tra Quota ngay từ đầu giúp ngăn chặn các hành vi gian lận hoặc sử dụng vượt mức cho phép của doanh nghiệp

4.4.3.2. Quy trình ký hóa đơn

Quy trình Ký hóa đơn là bước then chốt trong hệ thống hóa đơn điện tử, biến một hóa đơn nháp (Draft) thành hóa đơn có giá trị pháp lý thông qua chữ ký số. Quy trình này đảm bảo tính toàn vẹn dữ liệu theo thông tư TT78/2021/TT-BTC và chuẩn bị dữ liệu để gửi lên cơ quan thuế thông qua các nhà cung cấp dịch vụ (Provider).



Hình 15: Sơ đồ Tuần tự quy trình ký hóa đơn

Các thành phần tham gia

Bảng 2: Bảng các thành phần tham gia quy trình ký hóa đơn

Tên thành phần	Ý nghĩa
Client/Merchant	Người dùng hoặc hệ thống gửi yêu cầu ký hóa đơn.
API Gateway & Auth Service	Tiếp nhận yêu cầu, xác thực quyền sở hữu hóa đơn của Merchant.
InvoiceSigningService	Trung tâm điều phối logic ký số.
Repositories (InvRepo, ConfigRepo, PayloadRepo)	Các lớp truy xuất MariaDB để lấy thông tin hóa đơn, cấu hình chứng thư số và lưu trữ nội dung XM.
XML Generator	Thành phần chịu trách nhiệm tạo cấu trúc file XML theo đúng chuẩn kỹ thuật của Tổng cục Thuế.
HSM Service / USB Token Driver	Thành phần kỹ thuật thực hiện việc ký số (sử dụng thiết bị phần cứng bảo mật hoặc phần mềm.
InvoicesMetadata	Thực thể lưu trữ thông tin bao quát của hóa đơn.
InvoiceSyncQueue	Hàng chờ xử lý đồng bộ để gửi hóa đơn đã ký sang nhà cung cấp dịch vụ bên thứ ba (VNPT, Viettel, MISA...)

Chi tiết quy trình

Bước 1: Khởi tạo yêu cầu ký (Initialization). Khách hàng gửi yêu cầu ký tới API với ID hóa đơn cụ thể. Auth Service thực hiện kiểm tra (1) API Key có hợp lệ không? (2) Hóa đơn này có thực sự thuộc về doanh nghiệp (Merchant) đang yêu cầu hay không?

Bước 2: Truy xuất và kiểm tra trạng thái hóa đơn. InvoiceSigningService lấy dữ liệu đầy đủ của hóa đơn từ database.Ràng buộc nghiệp vụ: Hóa đơn chỉ được phép ký nếu đang ở trạng thái DRAFT. Nếu hóa đơn đã ký rồi hoặc đã hủy, hệ thống ném lỗi InvalidStateException.

Bước 3: Lấy cấu hình chứng thư số (Certificate). Hệ thống truy xuất bảng cấu hình để lấy thông tin chứng thư số của Merchant bao gồm: Dữ liệu chứng thư (cert_data), Số Serial và phương thức ký (sử dụng Cloud HSM hay USB Token vật lý).

Bước 4: Cập nhật trạng thái "Đang ký" (State Transition). Trạng thái hóa đơn được chuyển sang SIGNING. Việc này giúp ngăn chặn các yêu cầu ký trùng lặp hoặc chỉnh sửa dữ liệu hóa đơn trong khi quá trình ký đang diễn ra.

Bước 5: Tạo nội dung XML chuẩn (XML Generation). XML Generator đóng gói toàn bộ thông tin (Người bán, người mua, danh mục hàng hóa, tiền thuế) vào định dạng XML theo chuẩn TT78/2021.File XML chưa ký (Raw XML) được lưu vào bảng invoice_payloads để phục vụ việc đối chiếu dữ liệu trước khi ký.

Bước 6: Thực hiện ký số (Digital Signing). Đây là bước quan trọng nhất về mặt kỹ thuật, hệ thống hỗ trợ 2 kịch bản. Nếu dùng Cloud HSM hệ thống sẽ kết nối với thiết bị bảo mật phần cứng tập trung để ký bằng Private Key của doanh nghiệp.Nếu dùng USB Token sẽ sử dụng trình điều khiển (Driver) để ký trực tiếp.Kết quả trả về là chuỗi dữ liệu chữ ký số (Digital Signature).

Bước 7: Nhúng chữ ký và lưu trữ bản cuối. Chuỗi chữ ký số được nhúng vào file XML gốc để tạo thành file XML hoàn chỉnh có chữ ký.Hệ thống cập nhật lại bảng invoice_payloads với nội dung XML đã ký này.

Bước 8: Xác nhận ký thành công. Cập nhật trạng thái hóa đơn thành SIGNING_SUCCESS.Ghi nhận thời điểm ký thành công vào trường signed_at.

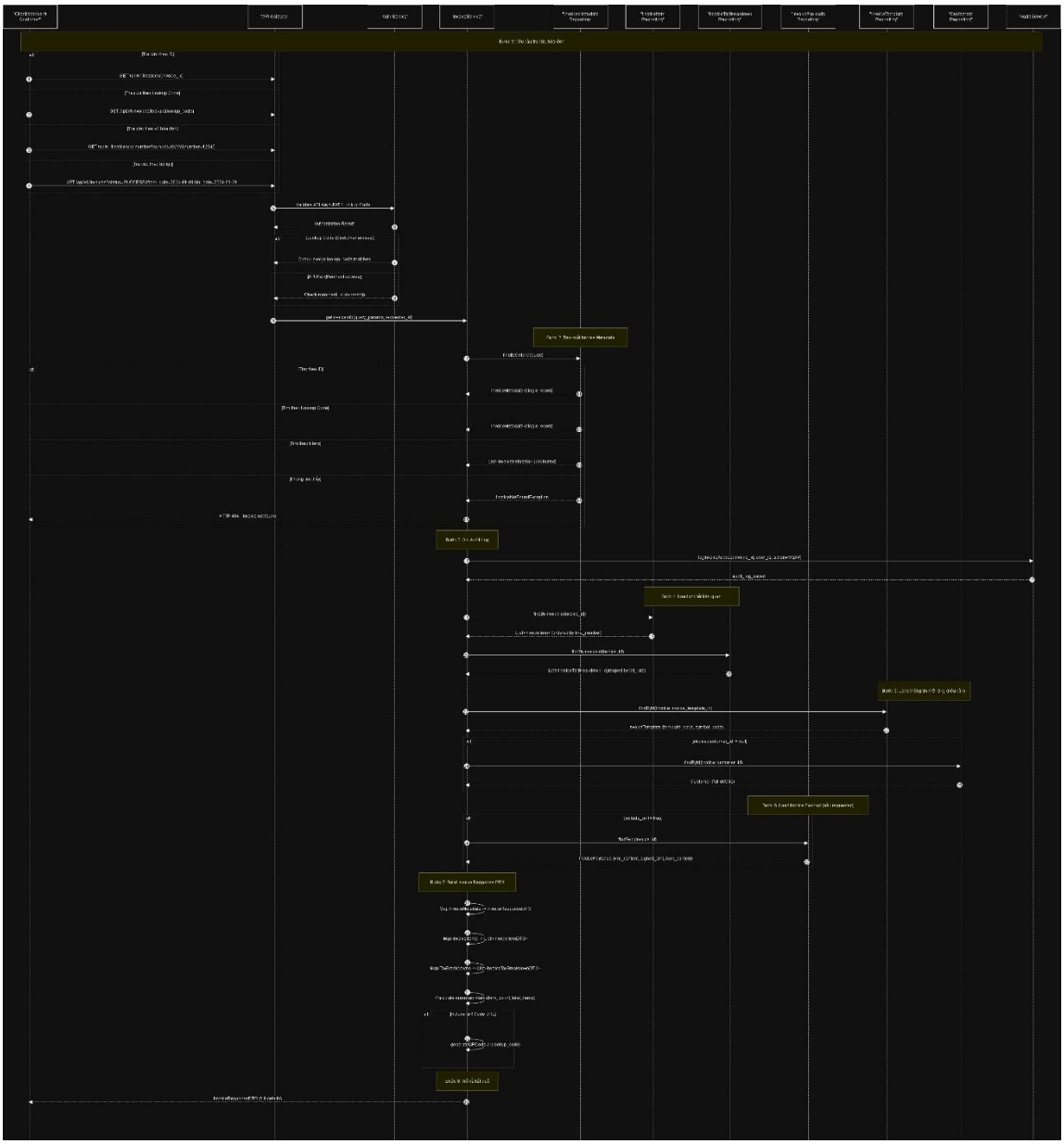
Bước 9: Đưa vào hàng chờ đồng bộ (Asynchronous Sync).Thay vì gửi trực tiếp lên Provider (như VNPT, Viettel) và bắt người dùng chờ đợi lâu, hệ thống đẩy hóa đơn đã ký vào InvoiceSyncQueue. Một tiến trình chạy ngầm (Background Worker) sẽ lấy dữ liệu từ hàng chờ này để thực hiện việc đẩy hóa đơn lên hệ thống của nhà cung cấp dịch vụ sau đó.

Bước 10: Trả về kết quả cho Client. Hệ thống phản hồi cho khách hàng thông tin xác nhận: Hóa đơn đã ký thành công, thời gian ký và trạng thái hiện tại

Các điểu thiết kế đáng kể là tính bảo mật qua việc kiểm tra quyền sở hữu hóa đơn (Ownership) chặt chẽ ở Bước 1. Tính linh hoạt trong hỗ trợ cả 2 hình thức ký phổ biến hiện nay là HSM (ký lô lớn, tự động) và Token vật lý. Tính ổn định khi sử dụng cơ chế hàng chờ (Queue) ở Bước 9 giúp hệ thống tránh bị treo nếu API của các nhà cung cấp (VNPT/Viettel) gặp sự cố chậm trễ. Tính pháp lý vì đảm bảo file XML được tạo ra tuân thủ nghiêm ngặt quy định của pháp luật về hóa đơn điện tử tại Việt Nam

4.4.3.3. Quy trình lấy thông tin hóa dơn

Quy trình "Lấy thông tin hóa đơn" cho phép các đối tượng khác nhau (Chủ doanh nghiệp, Kế toán, hoặc Khách hàng mua hàng) truy xuất dữ liệu chi tiết của một hoặc nhiều hóa đơn. Hệ thống hỗ trợ đa dạng phương thức tra cứu từ cơ bản đến nâng cao, đảm bảo tính bảo mật nghiêm ngặt thông qua phân quyền truy cập.



Hình 16: Sơ đồ Tuần tự quy trình lấy thông tin hóa đơn

Các thành phần tham gia

Bảng 3: Bảng các thành phần tham gia quy trình lấy thông tin hóa đơn

Tên thành phần	Ý nghĩa
Client (Merchant/Customer)	Người gửi yêu cầu tra cứu (Doanh nghiệp dùng API Key/JWT hoặc Khách hàng dùng Mã tra cứu).
API Gateway & Auth Service	Tiếp nhận, điều phối và xác thực quyền truy cập.
InvoiceService	Trung tâm điều phối logic, tổng hợp dữ liệu từ nhiều nguồn.

Repositories (InvRepo, ItemRepo, TaxRepo, PayloadRepo, TemplateRepo, CustomerRepo)	Các lớp truy xuất dữ liệu từ MariaDB để lấy thông tin định danh, chi tiết hàng hóa, phân rã thuế, nội dung XML và thông tin khách hàng.
Audit Service	Dịch vụ ghi nhật ký truy cập để theo dõi ai đã xem hóa đơn vào lúc nào.

Chi tiết quy trình

Bước 1: Yêu cầu tra cứu hóa đơn (Request Initiation). Hệ thống cung cấp 4 phương thức linh hoạt để khởi tạo yêu cầu. Tra cứu theo ID truy xuất trực tiếp nếu biết ID hệ thống. Tra cứu theo Mã tra cứu (Lookup Code) dành cho khách hàng lẻ không cần tài khoản, thường dùng để xem hóa đơn từ email/tin nhắn.Tra cứu theo Số hóa đơn tìm kiếm dựa trên cặp Ký hiệu (Symbol) và Số hóa đơn (Number).Tra cứu theo Bộ lọc (Filters) tìm kiếm danh sách hóa đơn theo trạng thái, khoảng thời gian (dành cho kế toán).Xác thực quyền nếu là Merchant (Doanh nghiệp), hệ thống kiểm tra API Key/JWT để đảm bảo họ chỉ xem được hóa đơn thuộc sở hữu của mình.Nếu là Customer (Khách hàng lẻ): Hệ thống kiểm tra sự trùng khớp của mã lookup_code.

Bước 2: Truy xuất Metadata (Thông tin định danh) InvoiceService gọi InvRepo để tìm bản ghi chính trong bảng invoices_metadata.Xử lý ngoại lệ nếu không tìm thấy dữ liệu tương ứng với tiêu chí, hệ thống ném lỗi InvoiceNotFoundException và phản hồi HTTP 404 cho người dùng ngay lập tức.

Bước 3: Ghi Nhật ký truy cập (Audit Log). Ngay khi tìm thấy hóa đơn, hệ thống gọi Audit Service để ghi lại hành động VIEW. Việc này cực kỳ quan trọng đối với các hệ thống hóa đơn chuyên nghiệp nhằm mục đích chống chối bỏ và kiểm soát an ninh thông tin.

Bước 4: Tải chi tiết hàng hóa và Phân rã thuế. Hóa đơn không chỉ có thông tin tổng quát mà cần các dữ liệu chi tiết. Invoice Items tải danh sách mặt hàng, đơn giá, số lượng (được sắp xếp theo thứ tự dòng hàng).Tax Breakdown tải thông tin tổng hợp thuế theo từng loại thuế suất (0%, 5%, 10%...) để hiển thị phần chân hóa đơn.

Bước 5: Tải thông tin mở rộng. Template Info truy xuất thông tin về mẫu số, ký hiệu hóa đơn từ bảng dải số. Customer Info nếu hóa đơn được

định danh cho một khách hàng có sẵn trong danh mục (Customer ID), hệ thống sẽ tải thông tin chi tiết của khách hàng đó.

Bước 6: Tải nội dung thô (Payload). Nếu người dùng yêu cầu xem dữ liệu kỹ thuật hoặc in ấn (include_xml = true), hệ thống sẽ truy xuất bảng invoice_payloads để lấy nội dung XML gốc, XML đã ký hoặc nội dung JSON. Đây là dữ liệu nặng nên chỉ được tải khi thực sự cần thiết.

Bước 7: Tổng hợp DTO (Data Transfer Object). Hệ thống thực hiện Mapping (ánh xạ) tất cả các thực thể (Entities) vừa tải được thành một đối tượng phản hồi duy nhất (InvoiceResponseDTO).Tính toán thêm các số liệu thống kê nhanh (tổng số dòng hàng, tổng số lượng quy đổi). QR Code tự động sinh URL hoặc dữ liệu QR Code dựa trên mã tra cứu để người dùng có thể quét và kiểm tra nhanh trên thiết bị di động.

Bước 8: Trả về kết quả. Trả về DTO hoàn chỉnh chứa đầy đủ thông tin từ metadata, hàng hóa, tiền thuế cho đến các thông tin kỹ thuật mở rộng

Các điểm trong thiết kế đó là về việc phân quyền đa tầng qua tách biệt rõ ràng luồng truy cập của doanh nghiệp (toàn quyền) và khách hàng lẻ (chỉ xem được hóa đơn cụ thể qua mã tra cứu).Tối ưu hiệu năng khi các dữ liệu chi tiết và dữ liệu nặng (Payload/XML) được tải theo cơ chế tùy chọn, tránh làm chậm hệ thống khi chỉ cần liệt kê danh sách hóa đơn đơn giản.Tuân thủ Audit vì mọi hành vi xem hóa đơn đều được lưu vết, đảm bảo đáp ứng các tiêu chuẩn về an toàn thông tin và quy định của Cơ quan Thuế. Trải nghiệm người dùng được hỗ trợ đa dạng tham số tìm kiếm, giúp việc tra cứu trở nên thuận tiện trong mọi tình huống nghiệp vụ.

4.4.4. Sơ đồ hoạt động

V. CÔNG NGHỆ VÀ MÔI TRƯỜNG PHÁT TRIỂN

5.1. Ngôn ngữ lập trình và Framwork

Trong bối cảnh hệ thống EInvoiceHub đòi hỏi khả năng xử lý giao dịch tài chính với độ chính xác tuyệt đối, tính sẵn sàng cao và khả năng xử lý đồng thời quy mô lớn, thì việc quyết định phương án lựa chọn các công nghệ là một chiến lược quan trọng. Đề tài tập trung ứng dụng các công nghệ hiện đại nhất trong chu kỳ hỗ trợ dài hạn để đảm bảo tính ổn định và bảo mật cho hạ tầng nghiệp vụ hóa đơn điện tử.

5.1.1. Java 21 – SpringBoot Framework

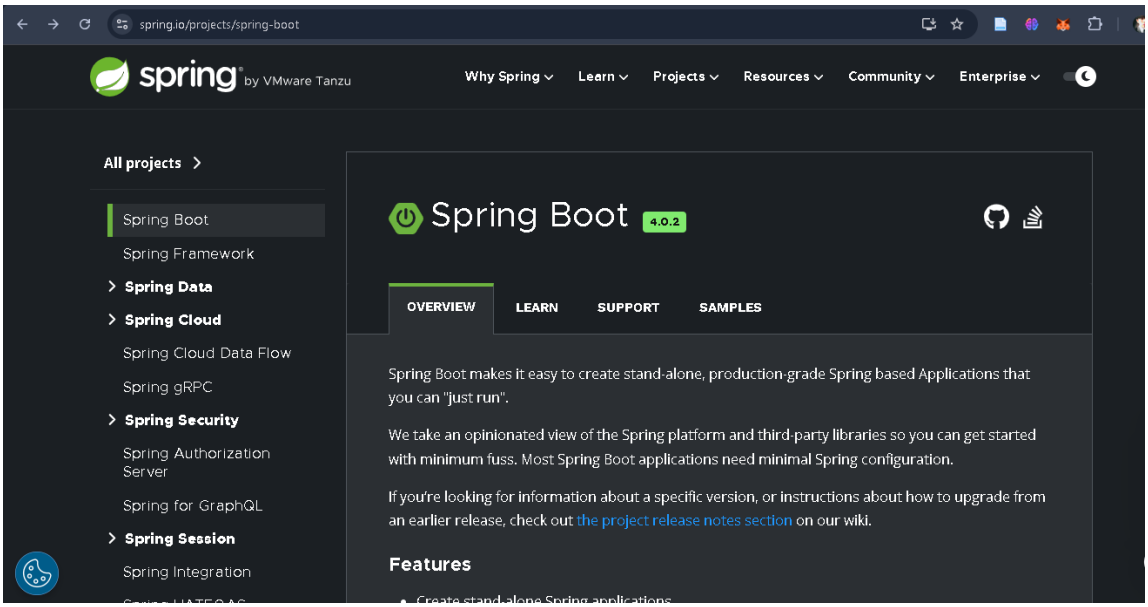
Java 21 là phiên bản hỗ trợ dài hạn mới nhất của ngôn ngữ Java, mang đến những cải tiến đột phá về hiệu suất thực thi và mô hình lập trình đồng thời. Sự lựa chọn Java 21 cho dự án EInvoiceHub không chỉ nằm ở tính kế thừa của một ngôn ngữ lập trình hướng đối tượng mạnh mẽ mà còn ở khả năng tối ưu hóa tài nguyên phần cứng thông qua luồng ảo Virtual Threads.

Hình 17: Tài liệu Oracle về Java 21

Về vai trò của Java 21, tài liệu chính thức từ Oracle khẳng định: Java 21 (Oracle JDK 21) mang đến hàng nghìn cải tiến về hiệu suất, độ ổn định và bảo mật, bao gồm các nâng cấp nền tảng giúp các nhà phát triển tăng năng suất, thúc đẩy đổi mới và tăng trưởng trong toàn bộ tổ chức.[7]

Tài liệu cũng khẳng định sự hỗ trợ dài hạn cho phiên bản này: Oracle sẽ cung cấp hỗ trợ dài hạn cho Java 21 trong ít nhất tám năm. Thời gian hỗ trợ kéo dài này mang lại cho các tổ chức sự linh hoạt để duy trì các ứng dụng trong môi trường sản xuất lâu hơn với mức bảo trì tối thiểu, và cuối cùng là chuyển đổi theo điều kiện riêng của họ.[7]

Bản phát hành này bao gồm hàng nghìn cập nhật về hiệu suất, độ ổn định và bảo mật, cùng với 15 JEP (Đề xuất cải tiến JDK) giúp cải thiện ngôn ngữ và nền tảng Java để giúp các nhà phát triển làm việc hiệu quả hơn. Đặc biệt, việc ứng dụng luồng ảo giúp EInvoiceHub giải quyết bài toán nghẽn I/O khi tương tác với các API của nhà cung cấp hóa đơn (TVAN) và ký số HSM. Oracle mô tả tính chất này như sau: Luồng ảo là các luồng nhẹ giúp giảm thiểu công sức khi viết, bảo trì và gỡ lỗi các ứng dụng đồng thời có thông lượng cao.[8]



Hình 18: Trang tài liệu Spring Boot

Kết hợp cùng ngôn ngữ Java, Spring Boot 3 đóng vai trò là khung phát triển chính (Framework) cho phía Backend. Spring Boot cung cấp một môi trường phát triển dựa trên nguyên lý Convention over Configuration, giúp đơn giản hóa việc cấu hình hạ tầng và tập trung vào logic nghiệp vụ hóa đơn. Theo định nghĩa từ trang chủ dự án: Spring Boot giúp dễ dàng tạo ra các ứng dụng độc lập, chất lượng cao dựa trên Spring mà bạn có thể "chạy ngay".Chúng tôi đưa ra quan điểm riêng về nền tảng Spring và các thư viện bên thứ ba để bạn có thể bắt đầu một cách dễ dàng nhất. Hầu hết các ứng dụng Spring Boot chỉ cần cấu hình Spring tối thiểu.[9]

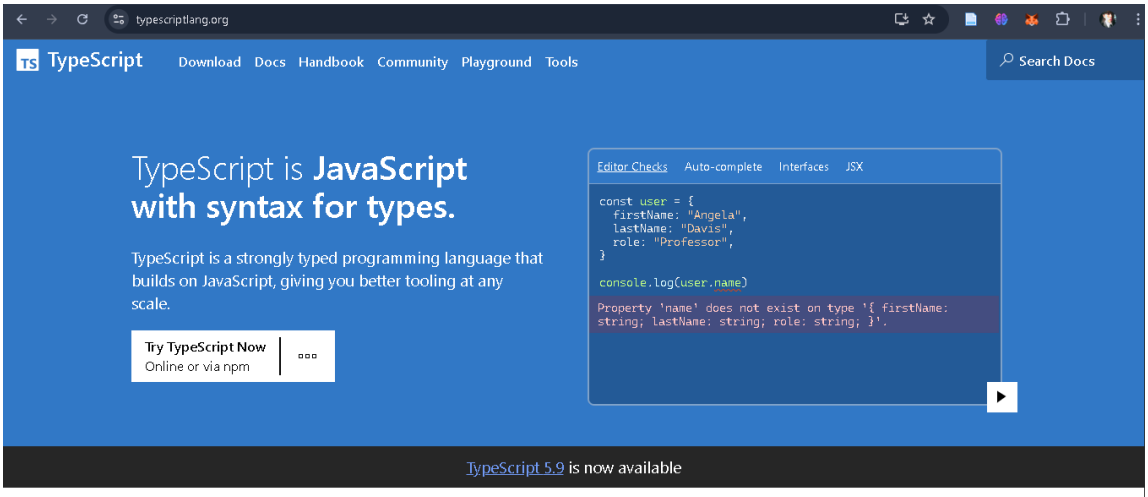
Một số đặc trưng của Springboot có thể kể đến như: Tạo các ứng dụng Spring độc lập. Nhúng trực tiếp Tomcat, Jetty hoặc Undertow (không cần triển khai các tệp WAR). Cung cấp các thư viện phụ thuộc "khởi đầu" có định hướng để đơn giản hóa cấu hình bản dựng của bạn.Tự động cấu hình Spring và các thư viện bên thứ ba bất cứ khi nào có thể.Cung cấp các tính năng sẵn sàng cho môi trường sản xuất, chẳng hạn như số liệu, kiểm tra trạng thái và cấu hình bên ngoài.Hoàn toàn không cần tạo mã và không yêu cầu cấu hình XML.[9]

Trong hệ thống, Spring Boot thực hiện điều phối toàn bộ luồng dữ liệu thông qua kiến trúc phân tầng, quản lý bảo mật với Spring Security và giao tiếp dữ liệu thông qua Spring Data JPA một cách nhất quán trên nền tảng MariaDB.

5.1.2. Type Script – Angolar Framework

Để xây dựng một giao diện quản trị hóa đơn điện tử chuyên nghiệp, có tính tương tác cao và đảm bảo tính nhất quán của dữ liệu từ tầng Client, đề tài sử dụng ngôn ngữ TypeScript kết hợp cùng khung phát triển Angular.

TypeScript được lựa chọn thay thế cho JavaScript truyền thống nhằm tận dụng cơ chế kiểm tra kiểu dữ liệu tĩnh (Static Typing), giúp hạn chế tối đa các lỗi logic trong quá trình phát triển ứng dụng Web quy mô lớn. Microsoft, đơn vị phát triển ngôn ngữ này, định nghĩa: TypeScript là một ngôn ngữ lập trình có kiểu dữ liệu mạnh mẽ, được xây dựng dựa trên JavaScript, cung cấp cho bạn các công cụ tốt hơn ở mọi quy mô.[10] . Tính chất này đặc biệt quan trọng khi xử lý các cấu trúc dữ liệu hóa đơn XML/JSON phức tạp, đảm bảo dữ liệu luôn đúng định dạng trước khi được gửi về Server.

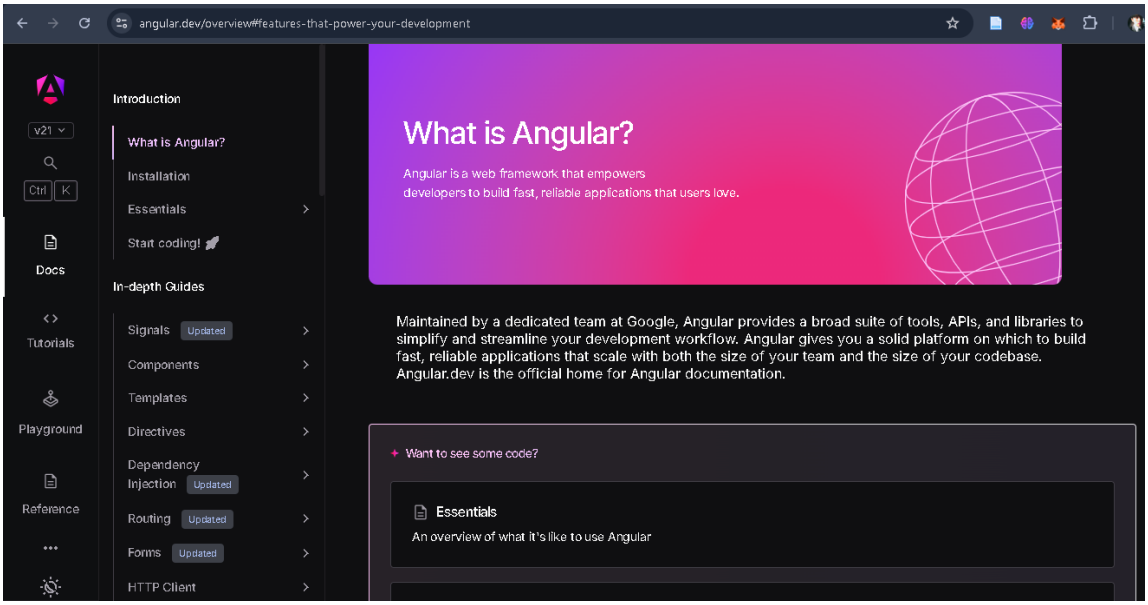


What is TypeScript?

Hình 19: Trang tài liệu về TypeScript

Angular, một nền tảng phát triển ứng dụng web phía khách hay Front-end do Google dẫn dắt, cung cấp một kiến trúc dựa trên thành phần mạnh mẽ gọi là Component-based. Angular giúp tách biệt rõ ràng giữa logic xử lý và hiển thị, tạo điều kiện cho việc bảo trì và mở rộng các tính năng như Dashboard thống kê hay Module quản lý Merchant. Tài liệu hướng dẫn của Angular nêu rõ: Angular là một framework web giúp các nhà phát triển xây dựng những ứng dụng nhanh, đáng tin cậy và được người dùng yêu thích. Được duy trì bởi một nhóm chuyên trách tại Google, Angular cung cấp một bộ công cụ, API và thư viện toàn diện để đơn giản hóa và tối ưu hóa quy trình phát triển của bạn. Angular cung cấp cho bạn một nền tảng vững chắc để xây dựng các ứng dụng nhanh, đáng tin cậy

và có khả năng mở rộng theo cả quy mô nhóm và quy mô mã nguồn của bạn. [11].



Hình 20: Trang tài liệu Angular

Angular là một nền tảng phát triển, được xây dựng trên TypeScript. Là một nền tảng, Angular bao gồm: Một khung dựa trên thành phần để xây dựng các ứng dụng web có khả năng mở rộng. Một bộ sưu tập các thư viện được tích hợp tốt bao gồm nhiều tính năng, bao gồm định tuyến, quản lý biểu mẫu, giao tiếp giữa máy khách và máy chủ, và nhiều tính năng khác. Nhờ vào cơ chế hai chiều hay là Two-way data binding và hệ thống RxJS xử lý các luồng dữ liệu bất đồng bộ, Angular cho phép EInvoiceHub cập nhật trạng thái hóa đơn thời gian thực và phản hồi nhanh chóng các thao tác của người dùng, mang lại trải nghiệm UI/UX tối ưu cho nhân viên kế toán và quản trị viên hệ thống.

5.2. Công cụ và môi trường phát triển

Để hiện thực hóa các yêu cầu kỹ thuật phức tạp của hệ thống EInvoiceHub, cần thiết lập một môi trường phát triển đồng nhất và lựa chọn các công cụ hỗ trợ tối ưu là yếu tố then chốt. Sự kết hợp giữa các môi trường lập trình tích hợp (IDE) hiện đại, hệ thống quản lý mã nguồn tập trung và công nghệ ảo hóa cấp độ điều hành giúp duy trì tính ổn định xuyên suốt từ giai đoạn lập trình đến triển khai thực tế.

5.2.1. IntelliJ Idea

Trong quá trình xây dựng tầng Backend dựa trên ngôn ngữ Java 21 và khung phát triển Spring Boot, dự án sử dụng IntelliJ IDEA làm môi trường phát triển tích hợp chính. Đây là công cụ được tối ưu hóa cho các

dự án doanh nghiệp lớn, cung cấp khả năng phân tích mã nguồn thông minh, hỗ trợ tái cấu trúc mã (Refactoring) mạnh mẽ và tích hợp sẵn các công cụ quản lý dự án như Maven, giúp đẩy nhanh tốc độ thực thi các nghiệp vụ hóa đơn phức tạp.

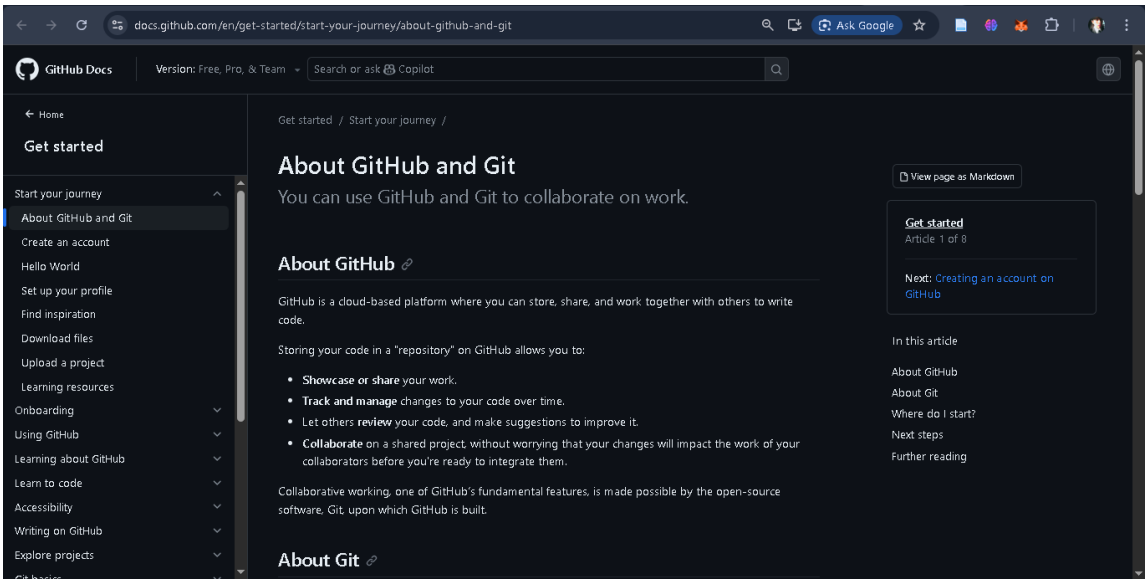
5.2.2. Visual Studio Code

Đối với tầng Frontend sử dụng Angular và TypeScript, Visual Studio Code được lựa chọn nhờ tính nhẹ nhàng và hệ sinh thái tiện ích (Extension) phong phú. VS Code hỗ trợ đắc lực trong việc kiểm soát cú pháp TypeScript theo thời gian thực và quản lý các luồng dữ liệu bất đồng bộ RxJS, đảm bảo giao diện quản trị luôn đạt hiệu suất cao và dễ dàng bảo trì.

5.2.3. Git và GitHub

Trong quản lý mã nguồn và điều phối quy trình phát triển, GitHub đóng vai trò là nền tảng lưu trữ và quản lý phiên bản dựa trên Git. GitHub không chỉ đơn thuần là kho chứa mã nguồn mà còn là hạt nhân của quá trình cộng tác và kiểm soát chất lượng phần mềm thông qua cơ chế Pull Request.

Theo tài liệu hướng dẫn chính thức từ trang chủ: Git là một hệ thống quản lý phiên bản giúp theo dõi các thay đổi trong tệp tin một cách thông minh. Git đặc biệt hữu ích khi bạn và một nhóm người cùng thực hiện các thay đổi trên cùng một tệp tin tại cùng một thời điểm.[12].



Hình 21: Tài liệu Git và GitHub

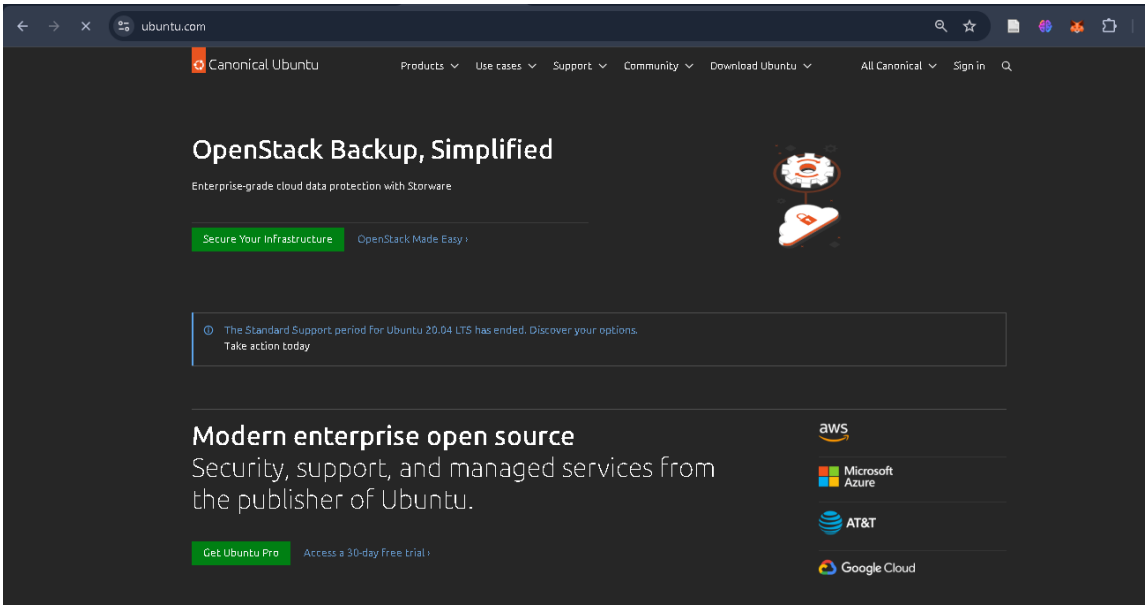
GitHub là một nền tảng dựa trên điện toán đám mây, nơi bạn có thể lưu trữ, chia sẻ và cộng tác với những người khác để viết mã. Việc lưu trữ mã nguồn của bạn trong một "kho lưu trữ" trên GitHub cho phép

bạn:Trưng bày hoặc chia sẻ tác phẩm của bạn. Theo dõi và quản lý các thay đổi đối với mã nguồn của bạn theo thời gian.Hãy để người khác xem xét mã của bạn và đưa ra những đề xuất để cải thiện nó.Cộng tác trong một dự án chung mà không cần lo lắng rằng những thay đổi của bạn sẽ ảnh hưởng đến công việc của các cộng tác viên trước khi bạn sẵn sàng tích hợp chúng.Làm việc cộng tác, một trong những tính năng cơ bản của GitHub, được thực hiện nhờ phần mềm mã nguồn mở Git, nền tảng mà GitHub được xây dựng dựa trên đó.[12]

Tại dự án EInvoiceHub, GitHub được tích hợp chặt chẽ với GitHub Actions để tự động hóa quy trình kiểm thử và đóng gói. Mỗi thay đổi về mã nguồn khi được đẩy lên hệ thống đều qua các bước kiểm tra tự động, giúp giảm thiểu rủi ro sai sót và đảm bảo rằng mã nguồn luôn trong trạng thái sẵn sàng để triển khai. Việc sử dụng GitHub còn giúp theo dõi lịch sử thay đổi chi tiết, phục vụ công tác đối soát và quản trị cấu hình hệ thống một cách khoa học.

5.2.4. Ubuntu

Hệ điều hành Ubuntu được chọn làm nền tảng máy chủ chính để vận hành các thành phần của hệ thống và các Node trong cụm Kubernetes. Ubuntu mang lại sự tin cậy tuyệt đối nhờ tính ổn định cao và khả năng tương thích hoàn hảo với các công cụ DevOps hiện đại.



Hình 22: Trang chủ Ubuntu

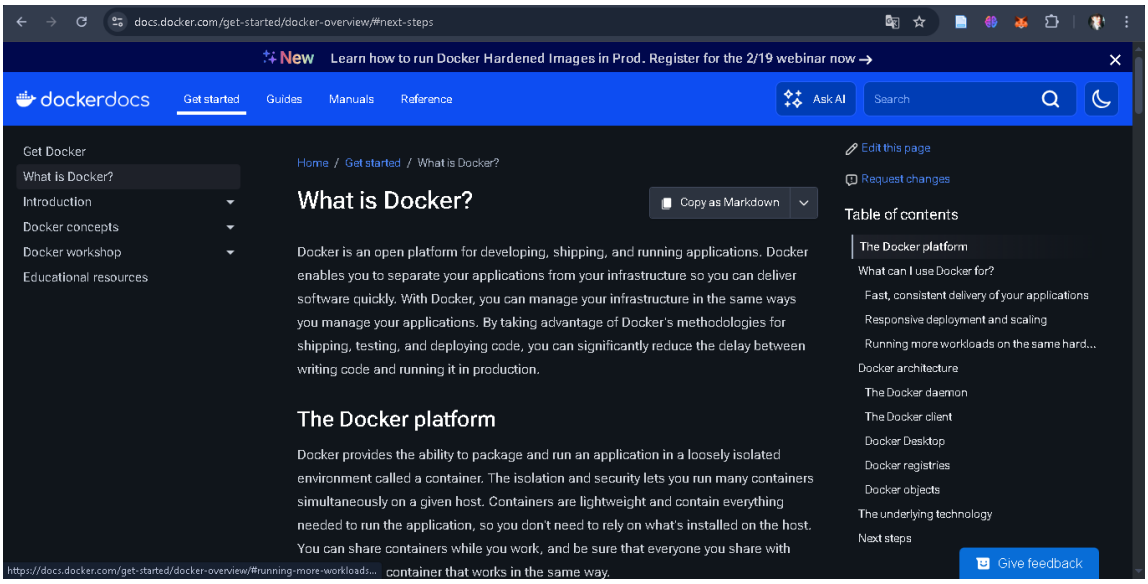
Nhà phát triển Canonical định nghĩa về hệ điều hành này như sau: Ubuntu là một hệ điều hành Linux hoàn chỉnh dành cho máy tính để bàn, được cung cấp miễn phí với sự hỗ trợ từ cộng đồng và giới chuyên nghiệp. Cộng đồng Ubuntu được xây dựng dựa trên những ý tưởng được nêu

trong Tuyên ngôn Ubuntu: phần mềm nên được cung cấp miễn phí, các công cụ phần mềm nên được người dùng sử dụng bằng ngôn ngữ địa phương của họ và bất kể khuyết tật nào, và người dùng nên có quyền tự do tùy chỉnh và thay đổi phần mềm theo bất kỳ cách nào họ thấy phù hợp.[13]

Sử dụng Ubuntu giúp hệ thống tận dụng tối đa khả năng quản lý tài nguyên của nhân Linux, đặc biệt là trong việc xử lý các luồng vào/ra (I/O) mật độ cao khi gọi API nhà cung cấp hóa đơn. Ngoài ra, cơ chế bảo mật nghiêm ngặt của Ubuntu với các bản cập nhật an ninh thường xuyên giúp bảo vệ hạ tầng EInvoiceHub khỏi các lỗ hổng tiềm ẩn, đảm bảo an toàn cho dữ liệu tài chính của doanh nghiệp.

5.2.5. Docker

Công nghệ container hóa Docker đóng vai trò trung tâm trong việc giải quyết bài toán đồng nhất môi trường triển khai cho EInvoiceHub. Docker cho phép đóng gói ứng dụng cùng toàn bộ các thư viện phụ thuộc vào một thực thể duy nhất, giúp loại bỏ hoàn toàn sự sai lệch giữa máy tính cá nhân và máy chủ vận hành. Tài liệu kỹ thuật của Docker mô tả: Docker là một nền tảng mở để phát triển, phân phối và chạy các ứng dụng. Docker cho phép bạn tách biệt ứng dụng khỏi cơ sở hạ tầng, nhờ đó bạn có thể cung cấp phần mềm nhanh chóng. Với Docker, bạn có thể quản lý cơ sở hạ tầng theo cùng một cách mà bạn quản lý ứng dụng của mình. Bằng cách tận dụng các phương pháp của Docker để phân phối, kiểm thử và triển khai mã, bạn có thể giảm đáng kể độ trễ giữa việc viết mã và chạy nó trong môi trường sản xuất [14].



Hình 23: Trang tài liệu Docker

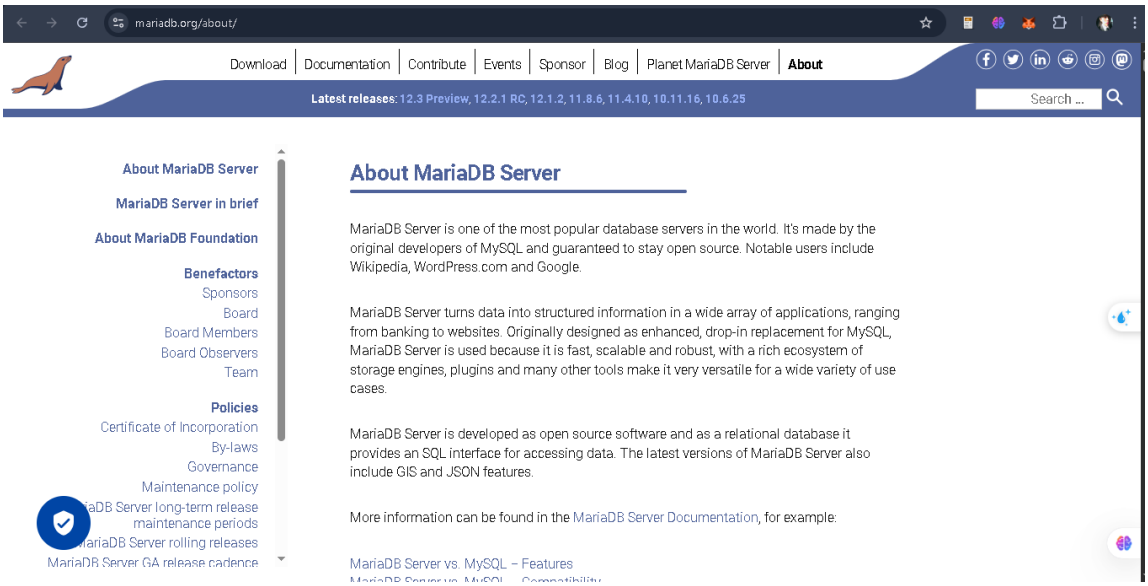
Thông qua Docker, các dịch vụ Backend, Frontend và cơ sở dữ liệu MariaDB được tách biệt thành các Container độc lập, giúp việc quản lý tài nguyên trở nên linh hoạt hơn. Đặc biệt, việc ứng dụng Docker Image giúp hệ thống có thể mở rộng nhanh chóng trên cụm Kubernetes, đảm bảo tính sẵn sàng cao khi lưu lượng phát hành hóa đơn tăng đột biến vào các kỳ báo cáo thuế cao điểm. Đây là nền tảng quan trọng để hiện thực hóa mô hình triển khai Cloud-Native cho toàn bộ dự án.

5.3. HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

Với kiến trúc hệ thống quản lý tài chính và hóa đơn điện tử của EinvoiceHub , tầng lưu trữ dữ liệu đóng vai trò là nền tảng cốt lõi, đảm bảo tính toàn vẹn, bảo mật và khả năng truy xuất nhanh chóng cho các giao dịch quan trọng. Cơ quan thực tập công ty cổ phần SOFTZ lựa chọn hệ quản trị cơ sở dữ liệu MariaDB làm cơ sở dữ liệu chính. Vì yêu cầu cao về hiệu năng và còn phải đáp ứng các tiêu chuẩn khắt khe về tính nhất quán trong môi trường đa người dùng.

5.3.1. MariaDB

MariaDB được lựa chọn làm hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) duy nhất cho toàn bộ hệ thống EInvoiceHub. Đây là một nhánh phát triển (fork) mã nguồn mở của MySQL, được xây dựng bởi các nhà phát triển nguyên thủy của MySQL với mục tiêu duy trì tính cộng đồng và cải thiện hiệu suất xử lý các truy vấn phức tạp.



Hình 24: Trang tài liệu MariaDB Server

Về bản chất và triết lý phát triển, trang chủ của tổ chức MariaDB Foundation đã khẳng định: MariaDB Server được phát triển như một phần mềm mã nguồn mở và là một cơ sở dữ liệu quan hệ, cung cấp giao diện SQL để truy cập dữ liệu. Các phiên bản mới nhất của MariaDB Server cũng bao gồm các tính năng GIS và JSON. MariaDB Server chuyển đổi dữ liệu thô thành thông tin có cấu trúc trong nhiều ứng dụng khác nhau, từ ngân hàng đến website. Ban đầu được thiết kế như một giải pháp thay thế nâng cao, dễ dàng tích hợp cho MySQL, MariaDB Server được sử dụng vì tốc độ nhanh, khả năng mở rộng và độ ổn định cao, cùng với hệ sinh thái phong phú gồm các công cụ lưu trữ, plugin và nhiều công cụ khác, giúp nó trở nên rất linh hoạt cho nhiều trường hợp sử dụng khác nhau.[15]

Trong dự án EInvoiceHub, MariaDB được cấu hình để lưu trữ dữ liệu hỗn hợp. Các bảng thông tin định danh và chỉ số tài chính được tổ chức chặt chẽ để phục vụ các truy vấn lọc và thống kê. Đồng thời, nội dung XML/JSON của hóa đơn có dung lượng lớn được lưu trữ dưới dạng LONGTEXT kết hợp với các kỹ thuật Indexing và Partitioning hợp lý. Giải pháp này cho phép hệ thống duy trì tốc độ phản hồi nhanh ngay cả khi quy mô dữ liệu đạt tới hàng triệu bản ghi, đồng thời đơn giản hóa việc triển khai và đồng bộ hóa trạng thái trên hạ tầng Kubernetes thông qua mô hình GitOps.

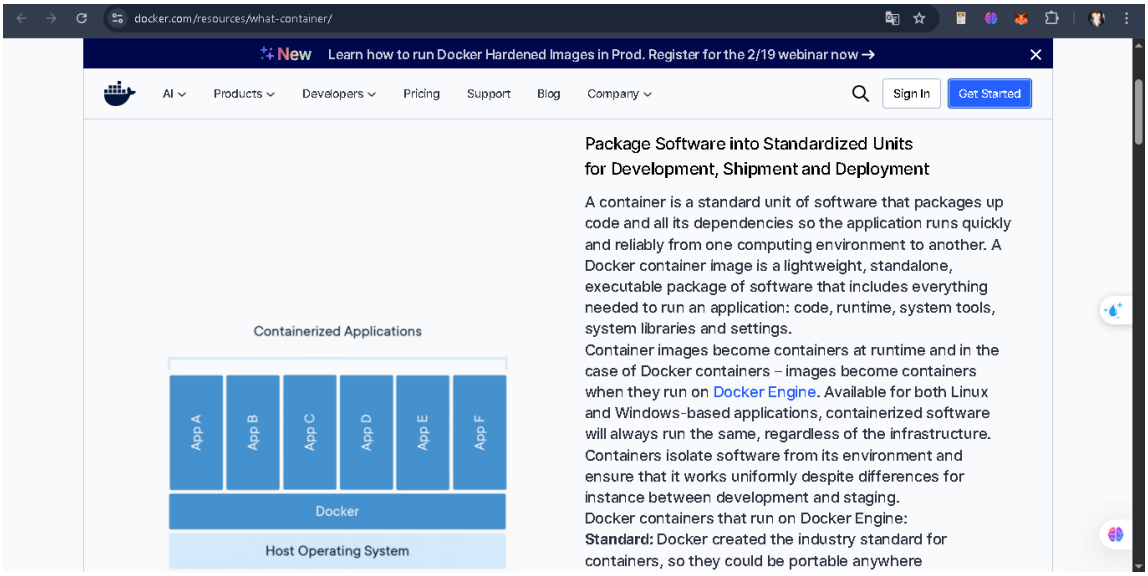
5.4. NỀN TẢNG TRIỂN KHAI VÀ TỰ ĐỘNG HÓA

Hiện nay, xu hướng phát triển phần mềm hiện đại đang dịch chuyển mạnh mẽ sang kiến trúc Cloud-native nhằm tối ưu hóa khả năng vận hành và khả năng mở rộng. Hệ thống EInvoiceHub không chỉ dừng lại ở việc hoàn thiện các tính năng nghiệp vụ mà còn tập trung xây dựng một hạ tầng triển khai tự động hóa hoàn toàn. Sự kết hợp giữa công nghệ đóng gói, điều phối hạ tầng và quy trình vận hành theo triết lý GitOps giúp hệ thống đạt được tính sẵn sàng cao và khả năng tự phục hồi trong môi trường sản xuất thực tế.

5.4.1. Công nghệ Containerization Docker

Công nghệ container hóa đóng vai trò là nền tảng trong việc giải quyết bài toán sai lệch môi trường và phiên bản giữa giai đoạn phát triển và triển khai. Docker cho phép đóng gói toàn bộ mã nguồn ứng dụng, các thư viện phụ thuộc và cấu hình hệ điều hành vào một thực thể duy nhất, đảm bảo tính nhất quán tuyệt đối khi vận hành trên bất kỳ hạ tầng nào.

Theo định nghĩa chính thức từ trang chủ Docker: Container là một đơn vị phần mềm tiêu chuẩn đóng gói mã nguồn và tất cả các phụ thuộc của nó để ứng dụng chạy nhanh và đáng tin cậy từ môi trường điện toán này sang môi trường điện toán khác. Docker Container Image là một gói phần mềm nhẹ, độc lập, có thể thực thi, bao gồm mọi thứ cần thiết để chạy một ứng dụng: mã nguồn, môi trường chạy, công cụ hệ thống, thư viện hệ thống và cài đặt.[16]

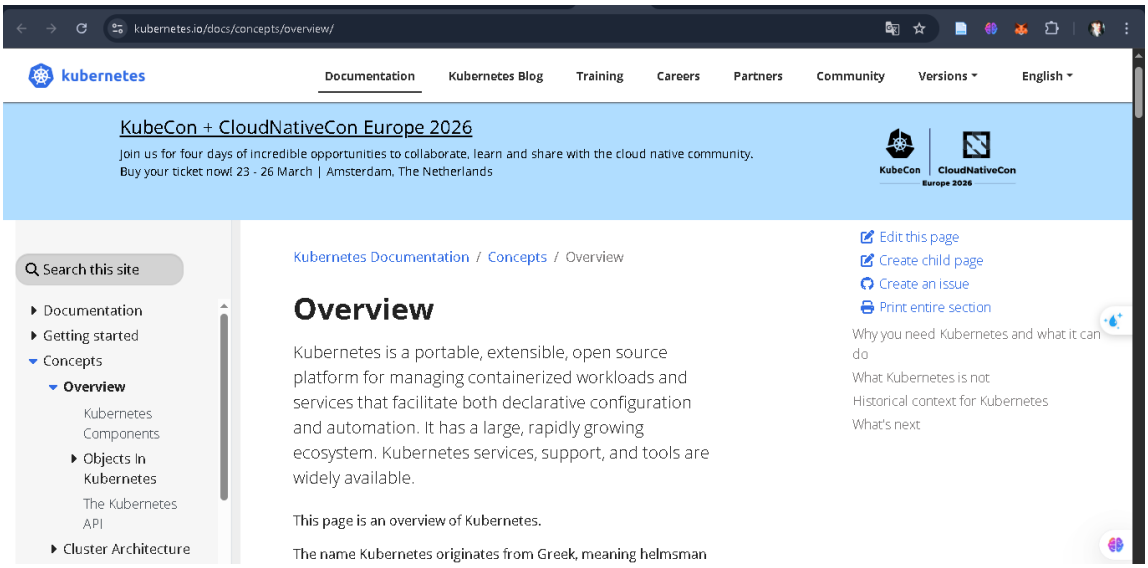


Hình 25: Trang tài liệu Docker giải thích về Docker Container và Docker Container Image

Trong dự án EInvoiceHub, mỗi thành phần bao gồm Backend Spring Boot, Frontend Angular và các dịch vụ hỗ trợ đều được đóng gói thành các Docker Image riêng biệt. Điều này không chỉ giúp việc quản lý phiên bản phần mềm trở nên minh bạch thông qua Docker Registry mà còn tạo tiền đề cho việc triển khai linh hoạt trên cụm điều phối Kubernetes. Việc cô lập tài nguyên của Docker cũng giúp bảo vệ các tiến trình xử lý hóa đơn nhạy cảm, tránh sự can thiệp từ các tiến trình khác trên cùng một máy chủ vật lý.

5.4.2. Điều phối hạ tầng Kubernetes và Terraform

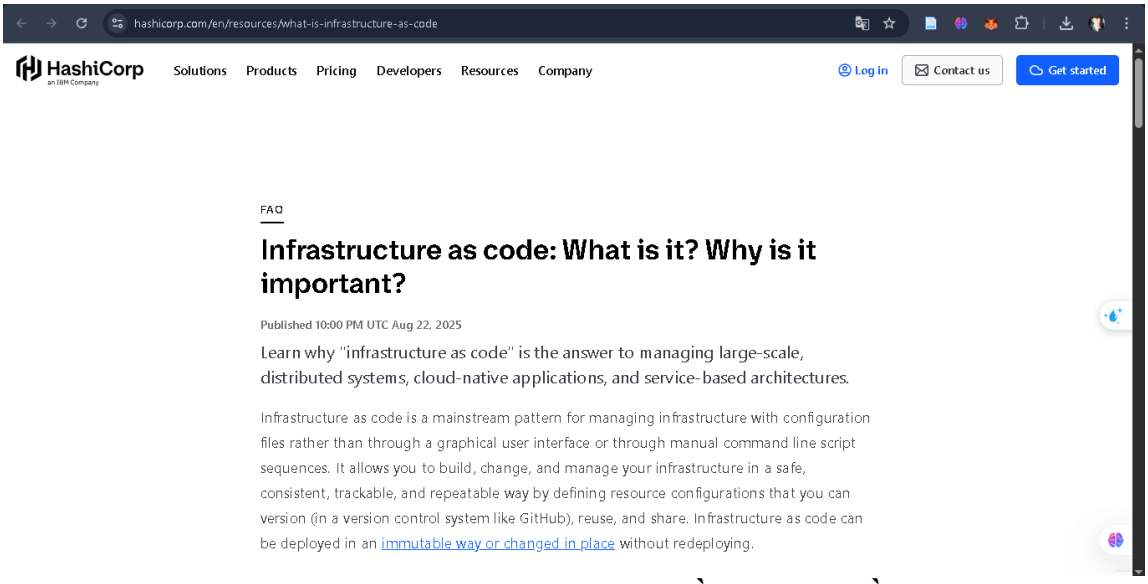
Để quản lý hàng loạt các container hoạt động đồng thời, dự án ứng dụng Kubernetes (K8s) làm hệ thống điều phối trung tâm. Kubernetes giúp tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng container hóa, đảm bảo rằng hệ thống EInvoiceHub luôn duy trì được số lượng bản sao cần thiết để đáp ứng lưu lượng truy cập từ các doanh nghiệp



Hình 26: Trang tài liệu Kubernetes

Tài liệu chính thức của Kubernetes mô tả về nền tảng này như sau: Kubernetes là một nền tảng mã nguồn mở, có tính di động và khả mở rộng, dùng để quản lý các khối lượng công việc và dịch vụ được đóng gói trong container, tạo điều kiện thuận lợi cho cả cấu hình khai báo và tự động hóa. Nó có một hệ sinh thái lớn và đang phát triển nhanh chóng. Các dịch vụ, hỗ trợ và công cụ của Kubernetes được cung cấp rộng rãi.[17]

Song song với việc điều phối container, dự án sử dụng Terraform để thực hiện triết lý Hạ tầng dưới dạng mã hay Infrastructure as Code - IaC. Terraform cho phép định nghĩa toàn bộ tài nguyên hạ tầng từ mạng VPC, máy ảo đến cụm Kubernetes thông qua các tập tin cấu hình có thể quản lý bằng phiên bản.



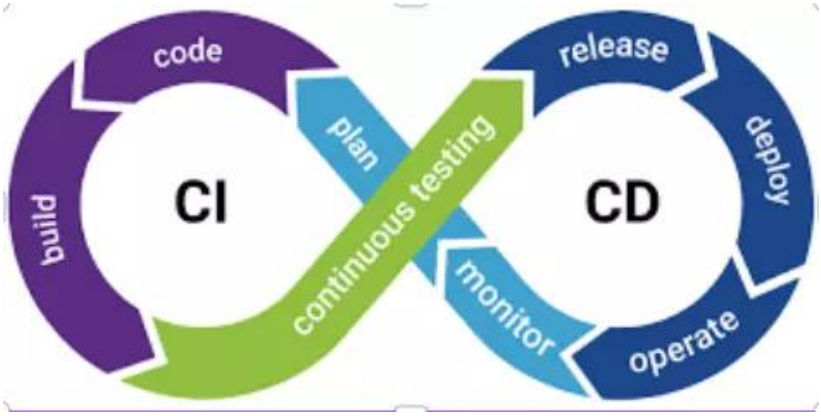
Hình 27: Trang tài liệu HashiCorp giải thích về Cơ sở hạ tầng dưới dạng mã

HashiCorp, đơn vị phát triển Terraform, định nghĩa: Cơ sở hạ tầng dưới dạng mã (Infrastructure as Code - ISD) là một mô hình phổ biến để quản lý cơ sở hạ tầng bằng các tệp cấu hình thay vì thông qua giao diện người dùng đồ họa hoặc các chuỗi lệnh thủ công. Nó cho phép bạn xây dựng, thay đổi và quản lý cơ sở hạ tầng của mình một cách an toàn, nhất quán, có thể theo dõi và lặp lại bằng cách định nghĩa các cấu hình tài nguyên mà bạn có thể quản lý phiên bản (trong hệ thống kiểm soát phiên bản như GitHub), tái sử dụng và chia sẻ. Cơ sở hạ tầng dưới dạng mã có thể được triển khai một cách bất biến hoặc thay đổi tại chỗ mà không cần triển khai lại.[18]

Nhờ có Terraform, việc thiết lập môi trường triển khai cho EInvoiceHub trở nên nhanh chóng, chính xác và loại bỏ hoàn toàn các sai sót do thao tác thủ công. Mọi thay đổi về hạ tầng đều được lưu vết và kiểm soát chặt chẽ, tạo nên một nền tảng kỹ thuật bền vững.

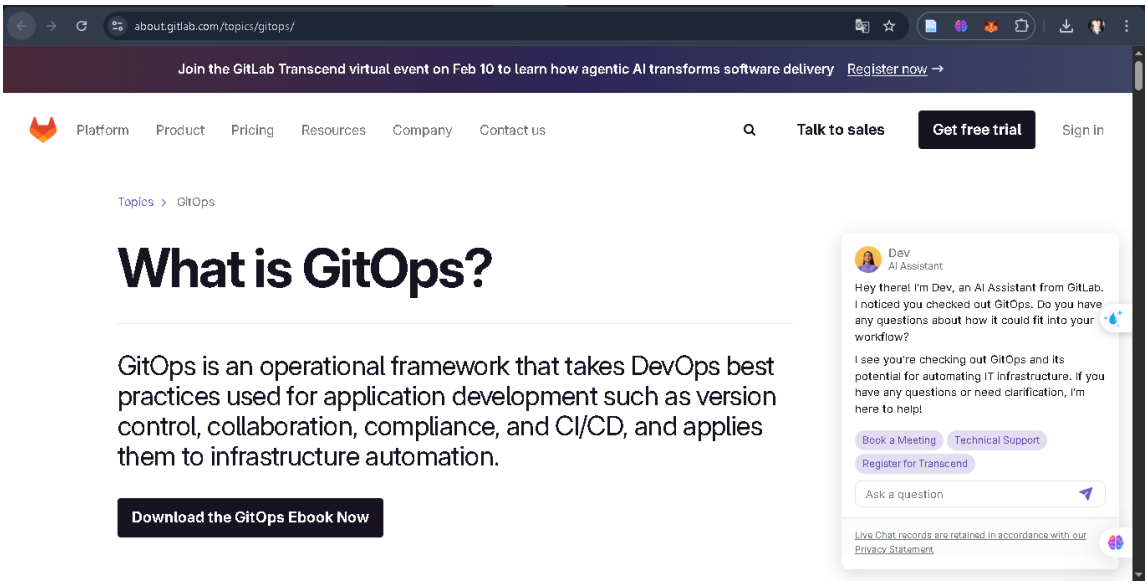
5.4.3. Quy trình triển khai hiện đại CI/CD và Triết lý GitOps với ArgoCD

Quy trình vận hành của EInvoiceHub xây dựng dựa trên sự kết hợp giữa tích hợp liên tục (CI) qua GitHub Actions và triển khai liên tục (CD) theo mô hình GitOps thông qua ArgoCD. GitOps là một phương thức vận hành hạ tầng và ứng dụng, trong đó Git đóng vai trò là "Nguồn sự thật duy nhất" (Single Source of Truth) cho toàn bộ trạng thái mong muốn của hệ thống.



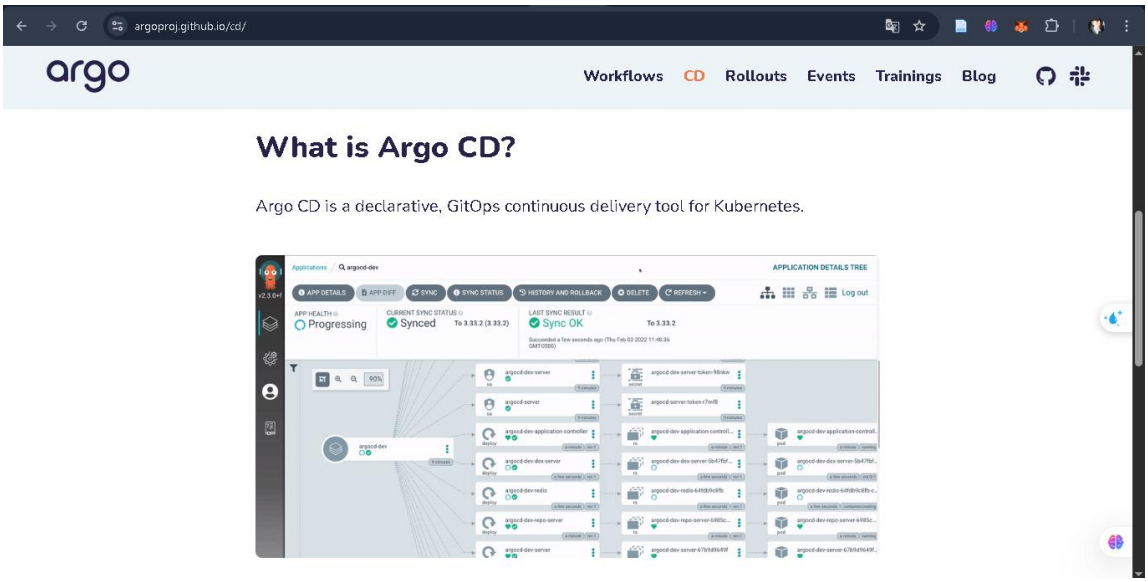
Hình 28: Minh họa quy trình CI/CD

Về khái niệm GitOps, nền tảng GitLab nêu rõ: GitOps là một khung vận hành lấy những thực tiễn tốt nhất của DevOps được sử dụng trong phát triển ứng dụng như kiểm soát phiên bản, hợp tác, tuân thủ và CI/CD, và áp dụng chúng vào tự động hóa cơ sở hạ tầng.[19]



Hình 29: Trang tài liệu GitLab

Trong luồng xử lý này, mỗi khi lập trình viên đẩy mã nguồn mới lên GitHub, GitHub Actions sẽ tự động thực hiện kiểm thử và đóng gói Docker Image. Sau đó, hệ thống sẽ cập nhật thông tin phiên bản mới vào kho lưu trữ cấu hình. Tại đây, ArgoCD đóng vai trò là tác nhân giám sát và đồng bộ.



Hình 30: Trang tài liệu Argo

Trang chủ dự án Argo CD mô tả tính năng của công cụ này: Argo CD là một công cụ phân phối liên tục dựa trên khai báo và GitOps dành cho Kubernetes. Việc định nghĩa ứng dụng, cấu hình và môi trường cần phải được khai báo và kiểm soát phiên bản. Việc triển khai ứng dụng và quản lý vòng đời cần được tự động hóa, có thể kiểm toán và dễ hiểu.[20]

Ứng dụng ArgoCD giúp hệ thống EInvoiceHub đạt được cơ chế tự đồng bộ trạng thái nếu có bất kỳ sự sai lệch nào giữa cấu hình trên cụm Kubernetes thực tế và cấu hình được định nghĩa trong Git, ArgoCD sẽ tự động điều chỉnh để đưa hệ thống về trạng thái mong muốn. Quy trình này không chỉ tăng cường tính bảo mật bằng cách hạn chế quyền truy cập trực tiếp vào cụm máy chủ mà còn giúp đội ngũ vận hành dễ dàng thực hiện quy trình hoàn tác (Rollback) khi có sự cố xảy ra, đảm bảo dịch vụ hóa đơn điện tử của doanh nghiệp hoạt động liên tục và ổn định

VI. TỔNG KẾT

6.1. KẾT LUẬN

6.1.1. Hoàn thành mục tiêu xây dựng giải pháp Middleware tập trung

Đề tài đã nghiên cứu và hiện thực hóa thành công hệ thống trung gian EInvoiceHub, giải quyết bài toán phân mảnh dữ liệu giữa các ứng dụng đầu cuối với các nhà cung cấp dịch vụ hóa đơn điện tử khác nhau. Thông qua việc thiết kế lớp trừu tượng hóa bằng các mẫu thiết kế Strategy và Adapter, hệ thống đã chuẩn hóa được luồng nghiệp vụ phát hành hóa đơn, cho phép doanh nghiệp vận hành linh hoạt mà không bị lệ thuộc vào giao diện lập trình riêng biệt của từng nhà cung cấp như MISA hay BKAV. Kết quả thực nghiệm cho thấy hệ thống đáp ứng đầy đủ các quy trình nghiệp vụ cốt lõi từ khởi tạo, ký số đến truyền nhận dữ liệu với Cơ quan Thuế theo đúng quy định tại Thông tư 78/2021/TT-BTC.

6.1.2. Hiệu quả của việc ứng dụng công nghệ Cloud-native và Java 21

Dự án đã khẳng định tính đúng đắn trong việc lựa chọn các công nghệ hiện đại để giải quyết bài toán hiệu năng và vận hành. Việc ứng dụng Java 21 với cơ chế Virtual Threads đã giúp hệ thống tối ưu hóa khả năng xử lý đồng thời, cho phép tiếp nhận hàng ngàn yêu cầu ký số và gọi API ngoại vi mà vẫn duy trì mức tiêu thụ tài nguyên thấp. Bên cạnh đó, việc triển khai hạ tầng dưới dạng mã bằng Terraform và vận hành theo triết lý GitOps trên nền tảng Kubernetes đã tạo ra một quy trình phát hành phần mềm tự động, minh bạch và có khả năng tự phục hồi cao. Hệ thống không chỉ dừng lại ở một ứng dụng nghiệp vụ mà còn là một mô hình hạ tầng công nghệ thông tin tiêu chuẩn, sẵn sàng cho việc mở rộng quy mô lớn trong tương lai.

6.2. HẠN CHẾ

6.2.1. Giới hạn về số lượng tích hợp nhà cung cấp và môi trường thử nghiệm

Trong thời gian thực hiện đề tài tốt nghiệp, hệ thống mới chỉ tập trung hoàn thiện và kiểm thử chuyên sâu đối với 02 nhà cung cấp dịch vụ hóa đơn điện tử phổ biến là MISA và BKAV. Dù kiến trúc Adapter cho phép mở rộng dễ dàng, thì việc tích hợp các đơn vị TVAN trên thị trường vẫn chưa được thực hiện do hạn chế về việc tiếp cận môi trường Sandbox và tài liệu kỹ thuật riêng biệt của từng đơn vị.

Ngoài ra, hầu hết các giao dịch hiện tại đang thực hiện trên môi trường thử nghiệm, dẫn đến việc chưa đánh giá được toàn diện các kịch bản lỗi phát sinh từ hệ thống thật của Tổng cục Thuế trong các giai đoạn cao điểm quyết toán.

6.2.2. Khó khăn trong việc tương tác với các thiết bị ký số vật lý

Hệ thống hiện tại tập trung tối ưu hóa cho hình thức ký số tập trung qua Cloud HSM, vốn là xu hướng của các doanh nghiệp lớn. Tuy nhiên, đối với các doanh nghiệp vừa và nhỏ vẫn đang sử dụng thiết bị USB Token vật lý, việc tích hợp quy trình ký số này vào môi trường Container trên Kubernetes vẫn gặp nhiều rào cản kỹ thuật về mặt nhận diện phần cứng và trình điều khiển. Điều này làm hạn chế khả năng tiếp cận của hệ thống đối với nhóm đối tượng người dùng có hạ tầng thiết bị truyền thống. Bên cạnh đó, việc phụ thuộc vào tính ổn định của API từ các bên thứ ba cũng là một rủi ro tiềm ẩn đối với tính liên tục của dịch vụ.

6.3. HƯỚNG PHÁT TRIỂN

6.3.1. Mở rộng hệ sinh thái tích hợp ứng dụng trí tuệ nhân tạo

Trong giai đoạn tiếp theo, hệ thống hướng tới việc tích hợp đầy đủ tất cả các nhà cung cấp dịch vụ hóa đơn điện tử tại Việt Nam và hỗ trợ thêm các chuẩn hóa đơn quốc tế. Hướng phát triển tiềm năng nhất là ứng dụng trí tuệ nhân tạo (AI) vào việc phân tích dữ liệu hóa đơn đã lưu trữ để cung cấp các báo cáo dự báo tài chính, tự động hóa việc phân loại chứng từ và phát hiện các dấu hiệu sai lệch thuế suất hoặc gian lận hóa đơn. Việc này sẽ chuyển đổi EInvoiceHub từ một cổng truyền nhận dữ liệu thuần túy thành một công cụ hỗ trợ ra quyết định thông minh cho bộ phận kế toán và quản trị doanh nghiệp.

6.3.2. Nâng cấp kiến trúc Microservices và bảo mật bằng Blockchain

Để đáp ứng nhu cầu xử lý khối lượng dữ liệu khổng lồ của các tập đoàn đa quốc gia, hệ thống cần được tái cấu trúc từ Monolithic-Core sang kiến trúc Microservices chuyên sâu, tách biệt hoàn toàn các dịch vụ ký

số, dịch vụ truyền nhận và dịch vụ tra cứu để có thể co giãn độc lập. Ngoài ra, nghiên cứu ứng dụng công nghệ Blockchain trong việc lưu vết nhật ký hệ thống (Audit Log) và mã tra cứu hóa đơn sẽ là hướng đi đột phá. Việc này giúp đảm bảo tính minh bạch, chống thay đổi dữ liệu tuyệt đối và tăng cường sự tin tưởng của các bên liên quan trong quy trình đối soát tài chính, đồng thời đưa EInvoiceHub tiệm cận với các tiêu chuẩn an toàn thông tin quốc tế.

VII. TÀI LIỆU THAM KHẢO

[1] "What Is Middleware? | IBM." Accessed: Feb. 10, 2026. [Online]. Available: https://www.ibm.com/think/topics/middleware

[2] "Middleware là gì?" Accessed: Feb. 10, 2026. [Online]. Available: https://www.redhat.com/en/topics/middleware/what-is-middleware

[3] "What is an API? (Application Programming Interface)," Mulesoft. Accessed: Feb. 10, 2026. [Online]. Available: https://www.mulesoft.com/api/what-is-an-api

[4] "foundation/charter.md at main · cncf/foundation," GitHub. Accessed: Feb. 10, 2026. [Online]. Available: https://github.com/cncf/foundation/blob/main/charter.md

[5] "2020-10-21 (1)." Accessed: Jan. 25, 2026. [Online]. Available: https://datafiles.chinhphu.vn/cpp/files/vbpq/2020/10/123.signed_01.pdf

[6] thuvienphapluat.vn, "Nghị định 70/2025/NĐ-CP sửa đổi Nghị định 123/2020/NĐ-CP hóa đơn chứng từ mới nhất," THƯ VIỆN PHÁP LUẬT. Accessed: Feb. 10, 2026. [Online]. Available: https://thuvienphapluat.vn/van-ban/Thue-Phi-Le-Phi/Nghi-dinh-70-2025-ND-CP-sua-doi-Nghi-dinh-123-2020-ND-CP-hoa-don-chung-tu-577816.aspx

[7] "Oracle Releases Java 21 and Extends Support Roadmap." Accessed: Feb. 09, 2026. [Online]. Available: https://www.oracle.com/news/announcement/ocw-oracle-releases-java-21-2023-09-19/

[8] "Virtual Threads," Oracle Help Center. Accessed: Feb. 09, 2026. [Online]. Available: https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html

[9] "Spring Boot," Spring Boot. Accessed: Feb. 09, 2026. [Online]. Available: https://spring.io/projects/spring-boot

[10] "JavaScript With Syntax For Types." Accessed: Feb. 09, 2026. [Online]. Available: https://www.typescriptlang.org/

[11] Team A., "What is Angular? • Angular." Accessed: Feb. 09, 2026. [Online]. Available: https://angular.dev/

[12] "About GitHub and Git," GitHub Docs. Accessed: Feb. 09, 2026. [Online]. Available: https://docs-internal.github.com/en/get-started/start-your-journey/about-github-and-git

[13] "system installation - What does a beginner need to know before switching to Ubuntu? - Ask Ubuntu." Accessed: Feb. 09, 2026. [Online]. Available: https://askubuntu.com/questions/49827/what-does-a-beginner-need-to-know-before-switching-to-ubuntu?newreg=8a7e794d4de64ed398e959e3abfe7f8b

[14] "What is Docker?," Docker Documentation. Accessed: Feb. 09, 2026. [Online]. Available: https://docs.docker.com/get-started/docker-overview/

[15] "About MariaDB Server," MariaDB.org. Accessed: Feb. 09, 2026. [Online]. Available: https://mariadb.org/about/

[16] "Container là gì? | Docker." Accessed: Feb. 09, 2026. [Online]. Available: https://www.docker.com/resources/what-container/

[17] "Overview," Kubernetes. Accessed: Feb. 09, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/

[18] HashiCorp, "What is infrastructure as code and why is it important?" Accessed: Feb. 09, 2026. [Online]. Available: https://www.hashicorp.com/en/resources/what-is-infrastructure-as-code

[19] "What is GitOps?," about.gitlab.com. Accessed: Feb. 09, 2026. [Online]. Available: https://about.gitlab.com/topics/gitops/

[20] "Argo CD." Accessed: Feb. 09, 2026. [Online]. Available: https://argoproj.github.io/cd/